Algorithmique Numérique pour l'Optimisation
(notes d'après cours)

François Boulier

10 novembre 2023

Table des matières

1 Introduction **3**
1.1 Relation avec les autres matières de la formation . . . 3
1.2 La reconnaissance des chiffres manuscrits . . . 3
1.3 Rappels d'algèbre linéaire . . . 5

2 Les moindres carrés linéaires **8**
2.1 L'exemple de la parabole . . . 8
2.2 Hypothèse d'indépendance linéaire . . . 9
2.3 La méthode historique . . . 9
2.4 La méthode moderne . . . 9
2.5 Exemple en Python . . . 10
2.6 Justifications . . . 11

3 La méthode de Newton en une variable **14**
3.1 Application à la résolution d'équation . . . 14
3.2 Exemples en Python . . . 15
3.3 La vitesse de convergence quadratique . . . 16
3.4 Application à la recherche de minima locaux . . . 18
3.5 Exemple en Python . . . 18

4 La méthode de Newton en plusieurs variables **20**
4.1 Fonctions de plusieurs variables . . . 20
4.2 Développement limité . . . 20
4.3 Écriture matricielle du développement limité . . . 21
4.4 Exemple . . . 23
4.5 Méthode de Newton pour résoudre un système carré . . . 23

5 Optimisation non contrainte **25**
5.1 Jacobienne, gradient et hessienne d'un objectif . . . 25
5.2 Méthode de Newton pour calculer un point stationnaire . . . 26
5.3 Exemple . . . 26
5.4 Un point stationnaire est un minimum local si la hessienne est définie positive . . . 27
5.5 Minima locaux, maxima locaux, cols . . . 28
5.6 Exemple . . . 28
5.7 Nécessité de contrôler la longueur du pas . . . 28

5.8 Le gradient donne la direction de la plus grande pente . . . 28
5.9 Algorithme du gradient . . . 29
5.10 Méthode de Newton avec contrôle du pas . . . 29
5.11 Contrôle de la longueur du pas . . . 30
5.12 Exemple . . . 30
5.13 Discussion . . . 31

6 Optimisation sous contrainte **33**
6.1 Le gradient est orthogonal aux courbes de niveau . . . 33
6.2 Multiplicateurs de Lagrange . . . 34
6.3 Lagrangien d'un problème d'optimisation sous contrainte . . . 35
6.4 Résolution d'un problème d'optimisation sous contrainte . . . 35
6.5 Exemple . . . 35
6.6 Forme générale d'un problème d'optimisation sous contrainte . . . 37
6.7 Exemples de problèmes d'optimisation sous contrainte . . . 37
6.7.1 Régression de ridge et Lasso . . . 37
6.7.2 Programmation linéaire . . . 38
6.8 Preuve de la proposition 21 . . . 39

7 Retour aux réseaux de neurones **41**
7.1 Un réseau à deux couches . . . 41
7.2 Un réseau à trois couches . . . 43
7.3 Commentaires . . . 44

8 Dérivation automatique **46**
8.1 Mode *forward* . . . 46
8.2 Mode *backward* . . . 47
8.2.1 Justification de la formule (8.1) . . . 49

Chapitre 1

Introduction

1.1 Relation avec les autres matières de la formation

Le cours d'Algorithmique Numérique pour l'Optimisation vise à clarifier certains mécanismes et points théoriques au cœur de plusieurs matières de la formation :

— Machine Learning : l'algorithme du gradient et les techniques de dérivation automatique ;

— Classification supervisée : la méthode de Newton qui est un concurrent de l'algorithme du gradient, est utilisée pour calculer le maximum de vraisemblance dans le cadre de la régression logistique ;

— Modélisation avancée : la technique des multiplicateurs de Lagrange, qui relève de l'optimisation sous contrainte, est employée pour formuler les objectifs de la régression de ridge ou celui du Lasso ;

— Recherche opérationnelle : les conditions de Karush, Kuhn et Tucker permettent de formuler l'objectif du simplexe (la résolution de programmes linéaires) sous la forme d'un problème d'optimisation sous contrainte. Ce problème peut se résoudre avec l'algorithme du point intérieur qui est aujourd'hui un concurrent sérieux du simplexe.

Toujours dans le but de motiver le cours, on présente une application très classique du *machine learning* (la reconnaissance des chiffres manuscrits) en parallèle avec la méthode des moindres carrés linéaires.

Je tiens à remercier les relecteurs de ce support de cours et, en particulier, Louis Jeronimo (IS2A5 en 2023) pour avoir repéré une dérivée partielle erronée.

1.2 La reconnaissance des chiffres manuscrits

Pour une excellente vidéo introductive, voir [3]. Il s'agit d'un exemple très classique (voir [1, section 11]). On a affaire à deux fonctions. La première est un réseau de neurones (le modèle en quelque sorte) destiné à reconnaître les chiffres manuscrits (voir Figure 1.1). En entrée, le réseau prend une image de $28 \times 28 = 784$ pixels, qui sont des réels compris entre 0 (pixel noir) et 1 (pixel blanc). En sortie, il produit un vecteur v de dimension 10. Un réseau idéal marcherait ainsi : si on lui donne en entrée l'image d'un chiffre trois manuscrit (par exemple) alors le vecteur v doit avoir toutes ses coordonnées à 0 sauf la troisième qui doit valoir 1. En pratique, on n'en demande pas tant : le vecteur v doit avoir toutes ses coordonnées proches de 0 sauf la troisième, qui doit être proche de 1.

En plus de la première couche (784 neurones) et de la dernière (10 neurones), le réseau choisi comporte deux couches de 16 neurones chacune (un neurone est une variable réelle). Chaque neurone d'une couche est connecté à chaque neurone de la suivante. Prenons l'exemple d'un des 16 neurones (appelons-le b) de la deuxième couche. Sa valeur est définie par une fonction

$$b \quad = \quad \sigma(w_1\, a_1 + w_2\, a_2 + \cdots + w_{784}\, a_{784} - \mathit{biais})\,,$$

où les a_i sont les 784 neurones de la première couche, les w_i et le biais sont des coefficients (des paramètres) et où $\sigma(z)$ est une fonction proche de 0 si z est inférieur à un seuil critique et qui est proche de 1 si z est supérieur à ce seuil. La valeur du seuil s'ajuste en modifiant la valeur du biais. On peut prendre pour σ une fonction dont le graphe est une sigmoïde (d'où son nom) et qui a l'avantage, en ce qui nous concerne, d'être une fonction différentiable.

Tout cela, c'est pour un neurone. Si on compte toutes les connexions du réseau, on trouve que cette fonction dépend de

$$m \quad = \quad 784 \times 16 + 16 \times 16 + 16 \times 10 + 16 + 16 + 10 \quad = \quad 13002$$

paramètres. Le problème posé consiste donc à trouver un jeu de 13002 paramètres qui réalise le mécanisme de reconnaissance de chiffres manuscrits.

On dispose pour cela de p « points expérimentaux » stockés dans une base de données. Chaque point est un couple formé de l'image d'un chiffre manuscrit (qui joue le rôle de x_i) et du vecteur de dimension 10 attendu (qui joue le rôle de y_i).

On peut maintenant introduire la deuxième fonction : un *objectif* à minimiser qui est une somme de carrés de *résiduels*

$$(13002\ \mathit{paramètres}) \quad \mapsto \quad \mathit{erreur} \quad = \quad r_1^2 + r_2^2 + \cdots + r_p^2\,.$$

Chaque résiduel r_i est la norme deux de la différence entre le vecteur calculé et le vecteur attendu :

$$r_i \quad = \quad \|\mathit{réseau}(\mathit{image}_i) - \mathit{vecteur\ attendu}_i\|_2$$

Le carré d'un résiduel est lui-même une somme de 10 carrés. L'erreur est donc une somme de $10\,p$ carrés !

Une importante différence avec les moindres carrés linéaires : la solution optimale du problème d'optimisation (le vecteur de 13002 paramètres qui minimise l'erreur) n'est pas solution d'un système d'équations linéaires. On peut s'en rendre compte assez facilement : si on essaie d'exprimer la valeur des neurones de la couche finale comme des fonctions des neurones de la couche initiale, on obtient des fonctions composées formées de trois niveaux de σ imbriqués. Et même si on supprimait les fonctions σ, on obtiendrait des polynômes de degré 3 en les paramètres : aucune chance d'aboutir à un système d'équations linéaires.

On a en fait affaire à un problème de moindres carrés *non linéaires*. Cette différence implique que le graphe de la fonction d'erreur comporte un très grand nombre de minima locaux. Il est aujourd'hui impossible de calculer le minimum global de cette fonction. Le mieux qu'on puisse faire, c'est appliquer un algorithme itératif (au sens du cours d'Algèbre Linéaire Numérique) qui construise une suite de vecteurs $u_0, u_1, \ldots, u_n$ (chaque vecteur u_i est de dimension 13002) dont on espère qu'elle converge vers un minimum local satisfaisant.

L'algorithme généralement utilisé pour cela est l'algorithme du gradient parce qu'il est moins coûteux à mettre en œuvre que la méthode de Newton (c'est important vu le nombre de variables) et qu'il semble moins sensible au choix du point de départ u_0 (on est réduit à choisir u_0 aléatoirement).

1.3 Rappels d'algèbre linéaire

Les rappels qui suivent sont utiles à la présentation des méthodes de résolution des problèmes de moindres carrés linéaires. Ils seront utiles aussi dans les cours suivants. La plupart de ces notions ont été vues en *Algèbre Linéaire Numérique*. Une bonne référence bibliographique : [4].

Définition 1 *Deux vecteurs sont orthogonaux si leur produit scalaire est nul.*

Définition 2 *Une matrice Q est* orthogonale *si elle est inversible et $Q^{-1} = Q^T$.*

Si Q est orthogonale on a $Q^T\,Q$ égale à l'identité. Les colonnes de Q sont donc des vecteurs orthogonaux deux-à-deux et tous de longueur (norme deux) égale à 1.

Proposition 1 *Partitionnons un vecteur x en deux sous-vecteurs y et z. Alors $\|x\|_2^2 = \|y\|_2^2 + \|z\|_2^2$.*

Preuve. Le carré d'une norme deux est une somme de carrés. □

Proposition 2 *On ne modifie pas la norme deux d'un vecteur x si on le multiplie par une matrice orthogonale Q. Dit autrement, $\|x\|_2 = \|Q\,x\|_2$.*

Preuve. Il suffit de montrer que les carrés des deux normes deux sont égaux. On a $\|x\|_2^2 = x^T\,x$. Et on a $\|Q\,x\|_2^2 = x^T\,Q^T\,Q\,x$ d'après la règle pour la transposée d'un produit. Comme Q est orthogonale, on a $\|Q\,x\|_2^2 = x^T\,x$. □

Proposition 3 *(inégalité de Cauchy-Schwartz)*
Soient u et v deux vecteurs de $\mathbb{R}^m$. Alors

$$|u^T\,v| \quad \leqslant \quad \|u\|_2\,\|v\|_2$$

avec égalité si u et v sont parallèles.

Définition 3 *Une matrice A est* symétrique *si elle est égale à sa transposée.*

Proposition 4 *Les valeurs propres d'une matrice symétrique sont réelles.*

Preuve. Toute matrice carrée A admet une factorisation de Schur [4, Lecture 24] $A = Q\,T\,Q^*$ où T est une matrice triangulaire et Q est une matrice *unitaire* (les matrices unitaires généralisent les matrices orthogonales dans le contexte des matrices complexes ; on note Q^* la tranposée-conjuguée de la matrice complexe Q, qui est égale à son inverse puisque Q est unitaire). On a $A^* = Q\,T^*\,Q^*$ d'après la régle sur la transposée(-conjuguée) d'un produit.
Supposons A symétrique. Alors $A = A^*$ et donc $T = T^*$. Il faut donc que T soit diagonale et que ses éléments diagonaux soient réels (les réels sont les seuls nombres complexes qui soient égaux à leur conjugué). Comme les éléments diagonaux de T sont les valeurs propres de A, on voit que les valeurs propres de A sont réelles. □

Proposition 5 *Soit A une matrice de dimension quelconque. Alors le produit $A^T\,A$ est toujours défini. Le résultat est une matrice symétrique.*

Preuve. Il suffit de montrer que $A^T\,A$ est égale à sa transposée. D'après la règle pour la transposée d'un produit de matrices on a $(A^T\,A)^T = A^T\,(A^T)^T = A^T\,A$. $\square$

Définition 4 *Soit A une matrice symétrique. A est* définie positive *si, quel que soit le vecteur x non nul, on a $x^T\,A\,x > 0$.*

Proposition 6 *Soit A une matrice symétrique. A est définie positive si et seulement si toutes ses valeurs propres sont strictement positives.*

Preuve. L'implication $\Rightarrow$ de gauche à droite. On suppose A définie positive, on considère une valeur propre λ quelconque de A et on montre $\lambda > 0$. Prenons un vecteur propre x (non nul) de A associé à λ. Alors $x^T\,A\,x$, qui est égal à $\lambda\,x^T\,x$, est du même signe que λ puisque $x^T\,x$ est une somme de carrés. Comme $x^T\,A\,x > 0$ on a $\lambda > 0$.

L'implication $\Leftarrow$ de droite à gauche. On suppose toutes les valeurs propres de A positives, on considère un vecteur x non nul quelconque et on montre $x^T\,A\,x > 0$. Comme A est symétrique, elle admet une factorisation $Q\,\Lambda\,Q^T$ avec Q orthogonale et Λ diagonale (les éléments diagonaux λ_i de Λ sont les valeurs propres de A). En appliquant la règle pour la transposée d'un produit on voit que $x^T\,A\,x = y^T\,\Lambda\,y$ où $y = Q^T\,x$. Notons y_i les coordonnées du vecteur y. Comme λ est diagonale, on a $y^T\,\Lambda\,y = \sum \lambda_i\,y_i^2$. Comme les λ_i sont supposés positifs cette expression est positive. $\square$

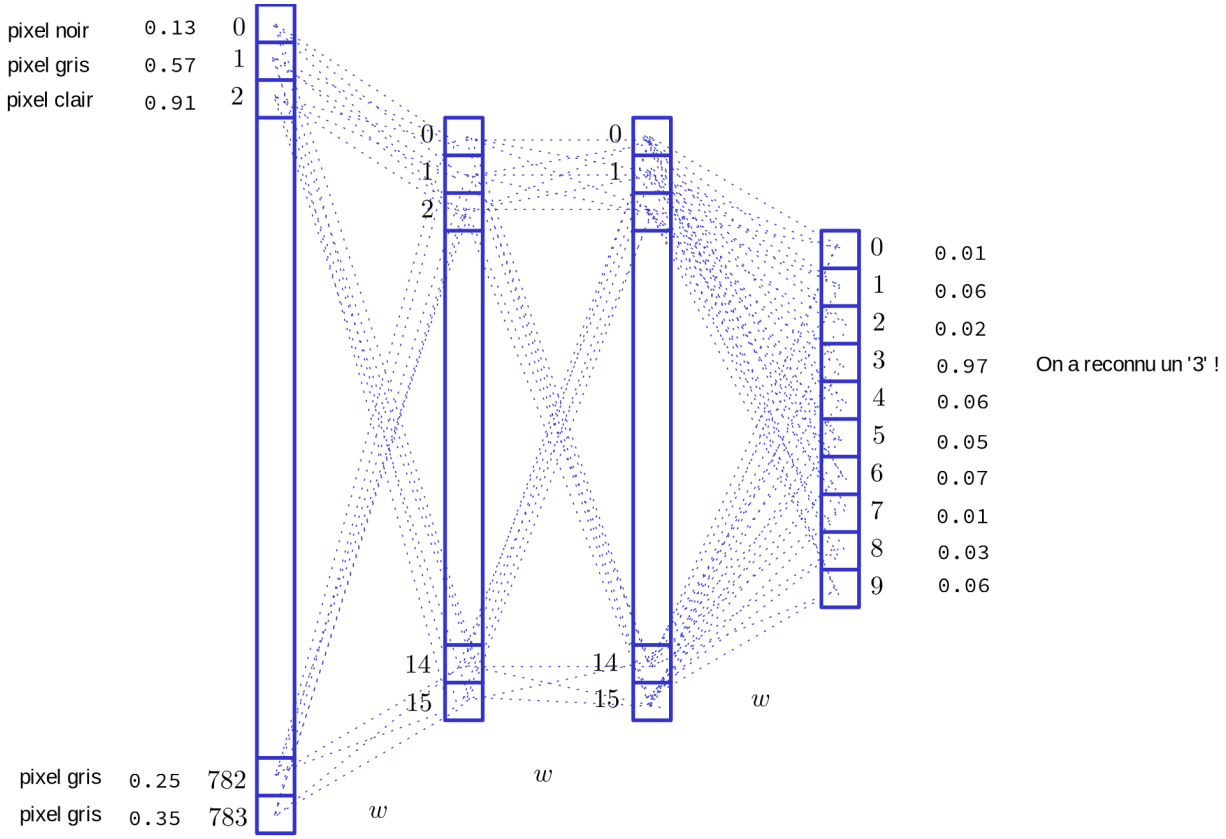


FIGURE 1.1 – Réseau de neurones dédié à la reconnaissance des chiffres manuscrits. La couche d'entrée, à gauche, contient une image de $28 \times 28 = 784$ pixels, qui sont des réels compris entre 0 (pixel noir) et 1 (pixel blanc). En sortie, à droite, un vecteur de 10 réels. Sur l'exemple, la coordonnée d'indice 3 du vecteur de sortie a une valeur nettement plus élevée que les autres. Ce résultat est interprété comme la reconnaissance du chiffre 3. En plus des couches d'entrée et de sortie, le réseau comporte deux couches intermédiaires de 16 neurones chacune. Chaque neurone d'une couche est fonction de tous les neurones de la couche précédente. Cette fonction dépend de poids w_i à raison d'un poids par trait en pointillé et d'un poids (le biais) par neurone des trois dernières couches. La phase d'entraînement du réseau de neurones a pour but : calculer des valeurs pour les poids afin que le réseau reconnaisse correctement les images de chiffres. En termes plus mathématiques, cette phase d'entraînement associe au réseau de neurones une fonction d'erreur (les variables de cette fonction d'erreur sont les poids du réseau) et cherche un minimum local de la fonction d'erreur. L'algorithme couramment utilisé est l'algorithme du gradient.

Chapitre 2

Les moindres carrés linéaires

2.1 L'exemple de la parabole

On illustre les méthodes sur l'exemple de la parabole. Elle est définie par une fonction $\mathbb{R} \to \mathbb{R}$ dépendant de $m = 3$ paramètres α, β et γ :

$$y \quad = \quad \alpha\, x^2 + \beta\, x + \gamma\,.$$

On dispose de p points expérimentaux (avec $p \geqslant m$) :

$$\begin{pmatrix} x_1 \\ y_1 \end{pmatrix}, \begin{pmatrix} x_2 \\ y_2 \end{pmatrix}, \ldots \begin{pmatrix} x_p \\ y_p \end{pmatrix}.$$

À chaque point d'indice i on associe un résiduel

$$r_i \quad = \quad \alpha\, x_i^2 + \beta\, x_i + \gamma - y_i$$

qui mesure l'écart vertical entre le point de la parabole d'abscisse x_i et le $i$ème point expérimental. L'objectif à minimiser est la (deuxième) fonction $\mathbb{R}^m \to \mathbb{R}$ qui, à un jeu de paramètres α, β et γ, associe la somme des carrés des résiduels :

$$(\alpha, \beta, \gamma) \quad \mapsto \quad r_1^2 + r_2^2 + \cdots + r_p^2\,.$$

Pour calculer la solution optimale du problème d'optimisation, on dispose de deux méthodes. Elles commencent toutes deux ainsi : on pose que la parabole passe par chacun des points [1] :

$$\left\{ \begin{array}{rcl} x_1^2\, \alpha + x_1\, \beta + \gamma & = & y_1\,, \\ x_2^2\, \alpha + x_2\, \beta + \gamma & = & y_2\,, \\ \vdots & \vdots & \vdots \\ x_p^2\, \alpha + x_p\, \beta + \gamma & = & y_p\,, \end{array} \right.$$

Ce même système peut s'écrire sous forme matricielle $A\, x = b$:

$$\underbrace{\begin{pmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ \vdots & \vdots & \vdots \\ x_p^2 & x_p & 1 \end{pmatrix}}_{A} \underbrace{\begin{pmatrix} \alpha \\ \beta \\ \gamma \end{pmatrix}}_{x} \quad = \quad \underbrace{\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_p \end{pmatrix}}_{b}.$$

1. Ce qui est un peu bizarre puisqu'on écrit un système dont on sait à l'avance qu'il n'a aucune solution.

Attention : à partir de maintenant, la lettre x désigne maintenant le vecteur des paramètres. Les symboles x_i continuent à désigner les ordonnées des points expérimentaux.

2.2 Hypothèse d'indépendance linéaire

Les deux méthodes présentées supposent que la matrice A soit de *rang maximal*. Comme on a aussi supposé $p \geqslant m$, cela revient à supposer que les p colonnes de la matrice A sont linéairement indépendantes, c'est-à-dire que A est de *rang* p.

Cette hypothèse est très naturelle sur l'exemple de la parabole : pour qu'elle ne le soit pas, il faudrait par exemple prendre des points expérimentaux ayant tous la même abscisse, ce qui serait pour le moins étrange. Mais dans certains problèmes réels la question est importante pour deux raisons :

— les asbcisses x_i pourraient être des fonctions compliquées de variables cachées et ces fonctions pourraient créer des dépendances linéaires ;

— même si la matrice A est de rang p, elle peut être très proche d'une matrice de rang inférieur à p, ce qui fait que les deux méthodes présentées marcheront mal (le résultat sera très imprécis). Cette distance de A à la matrice de rang inférieur à p la plus proche peut s'obtenir par un calcul de SVD : c'est la plus petite des *valeurs singulières* de A.

2.3 La méthode historique

Elle est due à Gauss.

Proposition 7 *La solution optimale x du problème d'optimisation est égale à la solution du système d'équations linéaires suivant (appelé* système des équations normales*) :*

$$A^T A x = A^T b .$$

La matrice $A^T A$ est de dimension $m \times m$. Les vecteurs x et $A^T b$ sont de dimension m.

La matrice $A^T A$ est symétrique. L'hypothèse d'indépendance linéaire des colonnes de A implique qu'elle est définie positive. On peut donc lui appliquer l'algorithme de Cholesky (1900) et calculer une matrice L telle que

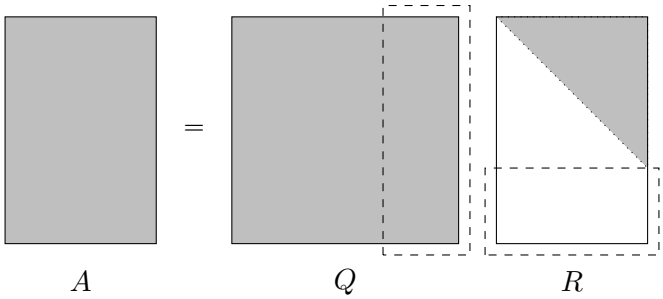
$$A^T A = L L^T .$$

Le système des équations normales peut donc se résoudre en enchaînant les deux résolutions de systèmes triangulaires suivantes :

$$\left\{ \begin{array}{rcl} L v & = & A^T b , \\ L^T x & = & v . \end{array} \right.$$

2.4 La méthode moderne

Elle date des années 1950 et repose sur la factorisation $Q R$ de la matrice A. Son gros avantage sur la méthode historique c'est d'éviter de former la matrice de covariance $A^T A$ dont les coefficients peuvent exploser lorsque p est grand.



La matrice Q est orthogonale. La matrice R est trapézoïdale supérieure. En partitionnant les matrices suivant les pointillés, on a la factorisation $Q\,R$ *complète* :

$$A \;=\; Q\,R \;=\; \begin{pmatrix} Q_1 & Q_2 \end{pmatrix} \begin{pmatrix} R_1 \\ 0 \end{pmatrix}.$$

Mais comme les lignes de R en dessous de R_1 sont nulles, la marice Q_2 ne contribue pas à R et on a également la factorisation $Q\,R$ *réduite* :

$$A \;=\; Q_1\,R_1\,.$$

On peut maintenant énoncer la proposition suivante

Proposition 8 *La solution optimale x du problème d'optimisation est égale à la solution du système d'équations linéaires suivant :*

$$R_1\,x \;=\; Q_1^T\,b\,.$$

La matrice R_1 est une matrice triangulaire supérieure de dimension $m \times m$. Les vecteurs x et $Q_1^T\,b$ sont de dimension m.

2.5 Exemple en Python

Voir le fichier `2-moindres_carres_lineaires.py`.

```
import numpy as np
import scipy.linalg as nla
import matplotlib.pyplot as plt
```

On se donne $p = 4$ points expérimentaux.

```
Tx = np.array ([2,-1,4,3], dtype=np.float64)
Ty = np.array ([15, -6.2, 58, 36.7], dtype=np.float64)
```

Le système d'équations $A\,x = b$ qui pose qu'une parabole passe par les points ($m = 3$).

```
b = Ty
A = np.array ([[x**2, x, 1] for x in Tx], dtype=np.float64)
```

On passe par la factorisation $Q\,R$ réduite de A.

```
Q1, R1 = nla.qr(A, mode='economic')
Q1Tb = np.dot (np.transpose (Q1), b)
alpha, beta, gamma = nla.solve_triangular (R1, Q1Tb, lower=False)
```

Vérification graphique. Voir figure 2.1.

```
def f(x) :
    return alpha*x**2 + beta*x + gamma

plt.scatter (Tx, Ty)
xplot = np.linspace (-1.1, 4.1, 50)
yplot = np.array ([f(x) for x in xplot])
plt.plot (xplot, yplot)
plt.show ()
```

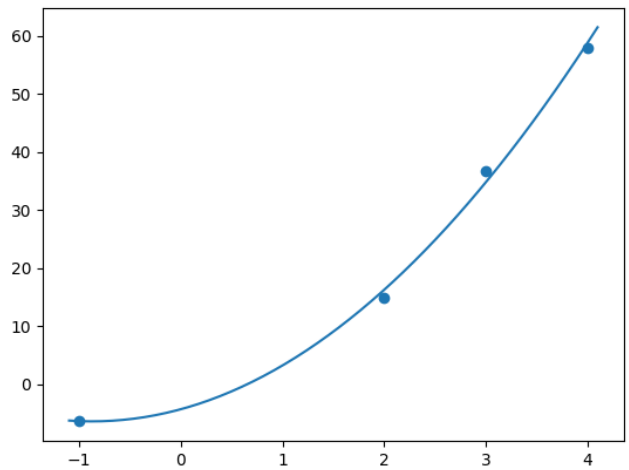


FIGURE 2.1 – Quatre points expérimentaux et la parabole qui passe au plus près d'eux, au sens des moindres carrés.

2.6 Justifications

Proposition 9 *La solution optimale x du problème d'optimisation est égale à la solution du système d'équations linéaires suivant :*

$$R_1\, x \quad = \quad Q_1^T\, b\,.$$

Preuve. La solution optimale x minimise la somme des carrés des résiduels. Cette erreur peut se formuler matriciellement : c'est le carré de la norme deux du vecteur $A\,x - b$:

$$\textit{erreur} \quad = \quad \|A\,x - b\|_2^2\,.$$

On ne change pas la norme deux d'un vecteur si on le multiplie par une matrice orthogonale. Et donc minimiser $\|A\,x - b\|_2^2$ est équivalent à minimiser $\|Q^T\,(A\,x - b)\|_2^2$ où Q provient de la factorisation $Q\,R$ de A. Or

$$\|Q^T\,(A\,x - b)\|_2^2 \quad = \quad \|Q^T\,Q\,R\,x - Q^T\,b\|_2^2 \quad = \quad \|R\,x - Q^T\,b\|_2^2\,.$$

Le carré d'une norme deux est une somme (de carrés), qu'on peut donc séparer en deux sous-sommes. On peut donc écrire

$$\|R\,x - Q^T\,b\|_2^2 \quad = \quad \|R_1\,x - Q_1^T\,b\|_2^2 + \|Q_2^T\,b\|_2^2$$

Le vecteur x qui annule $R_1\,x - Q_1^T\,b$ minimise le membre droit de l'égalité ci-dessus. Ce vecteur existe parce la matrice R_1 a tous ses éléments diagonaux non nuls (et est donc inversible). Cette propriété est une conséquence de l'hypothèse d'indépendance linéaire des colonnes de A. □

Proposition 10 *Le système $R_1\,x = Q_1^T\,b$ et le système des équations normales $A^T\,A\,x = A^T\,b$ ont même solution.*

Preuve. On a

$$A^T\,A \quad = \quad R^T\,Q^T\,Q\,R \quad = \quad R^T\,R$$

Le système des équations normales est donc équivalent au système

$$R^T\,R\,x \quad = \quad R^T\,Q^T\,b\,.$$

En tirant parti du partitionnement des matrices Q et R, on déduit que le système des équations normales est équivalent au système

$$\left\{ \begin{array}{ccc} R_1^T\,R_1\,x & = & R_1^T\,Q_1^T\,b\,, \\ 0 & = & 0\,. \end{array} \right.$$

L'hypothèse d'indépendance linéaire des colonnes de A implique que R_1^T est inversible. Le système des équations normales est donc équivalent au système

$$R_1\,x \quad = \quad Q_1^T\,b\,.$$

□

Proposition 11 *L'hypothèse d'indépendance linéaire des colonnes de A implique que la matrice symétrique $A^T\,A$ est définie positive.*

Preuve. Il suffit de considérer un vecteur x non nul quelconque et montrer que $x^T\,A^T\,A\,x > 0$. Or $x^T\,A^T\,A\,x = y^T\,y$ où $y = A\,x$. On a $A\,x = \xi_1\,a_1 + \xi_2\,a_2 + \cdots \xi_m\,a_m$ où $\xi_1, \xi_2, \ldots, \xi_m$ sont les coordonnées de x et $a_1, a_2, \ldots, a_m$ sont les colonnes de A. Comme ces dernières sont linéairement indépendantes et que x est non nul, le vecteur y est non nul. Le scalaire $y^T\,y$ est donc une somme de carrés strictement positive. □

Proposition 12 *L'hypothèse d'indépendance linéaire des colonnes de A implique que la matrice R_1 est inversible.*

Preuve. On a $A = Q_1 R_1$ (factorisation réduite de A). Notons r_{ij} l'élément de R_1 ligne i et colonne j. Notons a_j (resp. q_j) la $j$ème colonne de A (resp. Q_1). On a :

$$\begin{aligned} a_1 &= r_{11}\, q_1 \,, \\ a_2 &= r_{12}\, q_1 + r_{22}\, q_2 \,, \\ a_3 &= r_{13}\, q_1 + r_{23}\, q_2 + r_{33}\, q_3 \,, \\ &\vdots \end{aligned}$$

Supposons que r_{33} soit nul. Alors a_3 est combinaison linéaire de a_1 et de a_2. Ce raisonnement se généralise : si l'un des éléments diagonaux r_{ii} est nul, une colonne de A est combinaison linéaire des autres. Par conséquent (contraposée), si les colonnes de A sont linéairement indépendantes alors tous les éléments diagonaux sont non nuls et la matrice triangulaire R_1 est inversible. $\square$

Chapitre 3

La méthode de Newton en une variable

Le but de ce cours (plutôt élémentaire) : nous donner une intuition sur laquelle nous appuyer lorsque nous passerons au systèmes en plusieurs variables. La méthode de Newton (1700) a deux applications qui sont différentes mais liées :

1. résoudre une équation $f(x) = 0$,
2. minimiser un objectif $f(x)$.

On commence par la résolution d'équations.

3.1 Application à la résolution d'équation

Dans le cadre de la méthode de Newton, on suppose qu'on vise une racine α en particulier de la fonction f à résoudre et qu'on connaît une approximation u_0 de α.

La notion clé est la notion de développement limité.

$$f(u_0 + h) \quad = \quad f(u_0) + h\, f'(u_0) + h^2\, \frac{f''(u_0)}{2} + \cdots$$

Cela suppose que la fonction f soit suffisamment dérivable. Dans le cadre de la méthode de Newton, il est commode de supposer f deux fois dérivable. La méthode consiste à approximer la fonction f par son développement limité tronqué à l'ordre un et de chercher un déplacement h tel que $f(u_0+h) = 0$. On a

$$\begin{array}{rcll} 0 & = & f(u_0) + h\, f'(u_0) & \text{et donc} \\ h & = & -\frac{f(u_0)}{f'(u_0)} & \text{et donc} \\ u_0 + h & = & u_0 - \frac{f(u_0)}{f'(u_0)}. & \end{array}$$

En raison de l'approximation on a $f(u_0 + h)$ différent de zéro mais on peut se servir de $u_1 = u_0 + h$ comme nouvelle approximation et recommencer le calcul. Voir figure 3.1. La méthode consiste donc à calculer une suite de réels (u_n) dont on espère qu'elle converge vers α. Comme ce cours est destiné à préparer la présentation de la méthode de Newton en plusieurs variables, on reformule légèrement le calcul qui donne u_{n+1} en fonction de u_n :

$$\begin{array}{rrcl} \text{résoudre} & -f'(u_n)\, h & = & f(u_n)\,, \\ \text{prendre} & u_{n+1} & = & u_n + h\,. \end{array} \tag{3.1}$$

Remarque : la méthode de Newton calcule une suite de réels (u_n) mais on peut aussi considérer qu'elle calcule une suite de points $(x, y) = (u_n, f(u_n))$ sur le graphe $y = f(x)$.

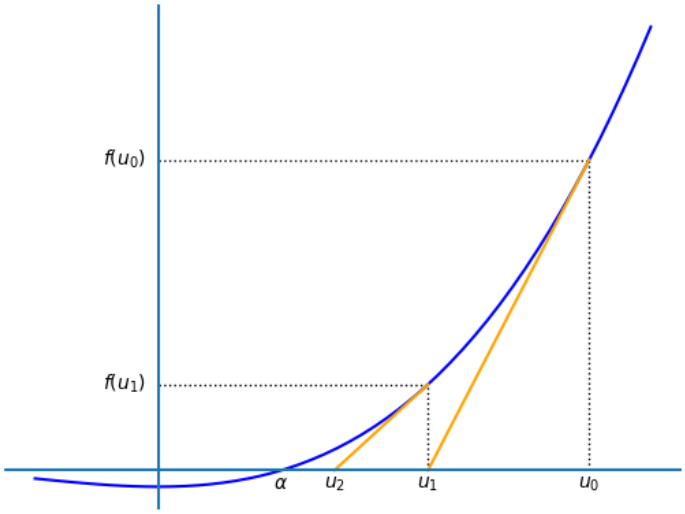


FIGURE 3.1 – Représentation graphique de la méthode de Newton.

3.2 Exemples en Python

On cherche le nombre 2 racine simple du polynôme $x^2 - 4$.

```
def f(x) :
    return x**2 - 4

def fprime(x):
    return 2*x

u = 4.
for n in range (0, 6):
    print ('u[%d] =' % n, u)
    u = u - f(u)/fprime(u)
```

La vitesse de convergence est quadratique : le nombre de décimales exactes double approximativement à chaque itération.

```
u[0] = 4.0
u[1] = 2.5
u[2] = 2.05
u[3] = 2.000609756097561
u[4] = 2.0000000929222947
u[5] = 2.000000000000002
```

On cherche le nombre 2 racine double du polynôme $(x - 2)^2$.

```
def f(x) :
    return (x-2)**2
```

```
def fprime(x):
    return 2*(x-2)

u = 4.
for n in range (0, 6):
    print ('u[%d] =' % n, u)
    u = u - f(u)/fprime(u)
```

La vitesse de convergence est linéaire : on gagne une décimale exacte toutes les trois ou quatre itérations.

```
u[0] = 4.0
u[1] = 3.0
u[2] = 2.5
u[3] = 2.25
u[4] = 2.125
u[5] = 2.0625
```

3.3 La vitesse de convergence quadratique

C'est elle qui explique pourquoi cette méthode, qui présente tant d'inconvénients (elle peut ne pas converger du tout, converger vers une autre racine que la racine α recherchée, ...) est si utilisée en pratique.

Proposition 13 *(vitesse de convergence quadratique)*
La vitesse de convergence de la méthode de Newton est quadratique en général, c'est-à-dire que le nombre de décimales exactes double (approximativement) à chaque itération.

Il est important de préciser cet énoncé. Pour commencer, on fait l'hypothèse[1] que la suite (3.1) converge bien la racine α désirée. On définit alors l'erreur à l'itération n par

$$e_n \quad = \quad u_n - \alpha \quad \text{ou encore} \quad u_n \quad = \quad e_n + \alpha\,. \tag{3.2}$$

On peut alors montrer que, si α est une racine simple de $f(x) = 0$ alors

$$\lim_{e_n \to 0} \frac{e_{n+1}}{e_n^2} \quad = \quad \text{cste}\,. \tag{3.3}$$

Si par contre α est une racine double (c'est-à-dire une racine qui annule non seulement f mais aussi sa dérivée f') alors la vitesse de convergence est la même que celle de la méthode dichotomique

$$e_{n+1} \quad = \quad \frac{e_n}{2}.$$

On prouve cette affirmation (dans le cas d'une racine simple) avec le paquetage `sympy` de Python.

1. L'analyse de la vitesse de convergence ne nous apprend donc rien sur ce qui se passe lorsque la méthode ne converge pas vers la racine désirée.

```
>>> import sympy as sp
>>> a, b, c, n, u, e = sp.symbols ('a, b, c, n, u, e')
>>> u = sp.IndexedBase(u)
>>> e = sp.IndexedBase(e)
```

On suppose que la fonction f est un polynôme de degré 2 ce qui revient à s'autoriser à approximer un fonction quelconque par un arc de parabole.

```
>>> f = a*u[n]**2 + b*u[n] + c
>>> f
a*u[n]**2 + b*u[n] + c
```

On introduit la dérivée de f ainsi qu'une racine α (peu importe laquelle). On remarque que la formule pour la racine suppose qu'elle soit simple.

```
>>> df = sp.diff (f, u[n])
>>> df
2*a*u[n] + b
>>> alpha = sp.solve(f,u[n])[0]
>>> alpha
(-b + sqrt(-4*a*c + b**2))/(2*a)
```

Voici la récurrence (3.1).

```
>>> unp1 = u[n] - f / df
>>> print (u[n+1], '=', unp1)
u[n + 1] = u[n] - (a*u[n]**2 + b*u[n] + c)/(2*a*u[n] + b)
```

On porte maintenant la dynamique sur l'erreur e_n : on utilise les relations (3.2) en définissant $e_{n+1} = u_{n+1} - \alpha$ et en remplaçant u_n par $e_n + \alpha$ dans toute la formule. On obtient une expression (abjecte) qui exprime e_{n+1} en fonction de e_n.

```
>>> enp1 = unp1 - alpha
>>> print (e[n+1], '=', enp1)
e[n + 1] = u[n] - (a*u[n]**2 + b*u[n] + c)/(2*a*u[n] + b) - (-b + sqrt(-4*a*c + b**2))/(2*a)
>>> enp1 = enp1.subs (u[n], e[n] + alpha)
>>> print (e[n+1], '=', enp1)
e[n + 1] = e[n] - (a*(e[n] + (-b + sqrt(-4*a*c + b**2))/(2*a))**2 + b*(e[n] + (-b +
           sqrt(-4*a*c + b**2))/(2*a)) + c)/(2*a*(e[n] + (-b + sqrt(-4*a*c + b**2))/(2*a)) + b)
```

Il se trouve que cette formule se simplifie ! On trouve

```
>>> enp1 = sp.simplify (enp1)
>>> print (e[n+1], '=', enp1)
e[n + 1] = a*e[n]**2/(2*a*e[n] + sqrt(-4*a*c + b**2))
```

De façon plus lisible, on a trouvé que

$$e_{n+1} \quad = \quad \frac{a\, e_n^2}{2\, a\, e_n + \sqrt{b^2 - 4\, a\, c}}. \tag{3.4}$$

La constante a est supposée non nulle puisque f est supposée de degré 2. On voit que si e_n tend vers zéro alors le rapport e_{n+1}/e_n^2 tend bien vers une constante d'autant plus petite que le discriminant $b^2 - 4\, a\, c$ est grand ou (ce qui revient au même) que les deux racines sont éloignées l'une de l'autre.

3.4 Application à la recherche de minima locaux

Pour trouver un minimum local d'une fonction $f(x)$ d'une variable réelle on procède en deux temps. On cherche d'abord les points $(x^*, f(x^*))$ du graphe de la fonction qui ont une tangente horizontale, c'est-à-dire tels que $f'(x^*) = 0$. Ensuite, pour chacun d'eux, on détermine s'il s'agit d'un minimum local en étudiant la dérivée seconde : si $f''(x^*) > 0$ alors $(x^*, f(x^*))$ est un minimum local. La raison se comprend bien en considérant le développement limité de la fonction f en un point x^* qui annule la dérivée :

$$f(x^* + h) \quad = \quad f(x^*) + \underbrace{f'(x^*)\, h}_{=0} \quad + \quad \underbrace{\frac{1}{2} f''(x^*)\, h^2}_{\substack{\text{même signe} \\ \text{que } f''(x^*)}} \quad + \cdots$$

La variable h représente un déplacement par rapport à x^* : si $h > 0$ on se déplace vers la droite ; si $h < 0$ on se déplace vers la gauche. Supposons $f''(x^*) > 0$. Comme $h^2 > 0$ quel que soit $h \neq 0$ on voit que $f(x^* + h) > f(x^*)$ quel que soit h non nul suffisamment petit en valeur absolue.

Pour trouver un point $(x^*, f(x^*))$ qui annule la dérivée de f il suffit d'appliquer la méthode de Newton à la fonction f' plutôt qu'à la fonction f.

3.5 Exemple en Python

On cherche un minimum local d'un polynôme de degré trois en appliquant la méthode de Newton sur la dérivée première du polynôme. Voir figure 3.2 pour le résultat.

```
alpha, beta, gamma, mu = 1.5870106105525719, \
    -3.1447447637103476, \
    -0.37473299354311346, \
    7.484053576868045

def f(x):
    return alpha*x**3 + beta*x**2 + gamma*x + mu

def fprime(x) :
    return 3*alpha*x**2 + 2*beta*x + gamma

def fseconde (x) :
    return 6*alpha*x + 2*beta

u = 0.8
for n in range (10) :
    print ('u[%d] =' % n, u)
    u = u - fprime(u)/fseconde(u)
```

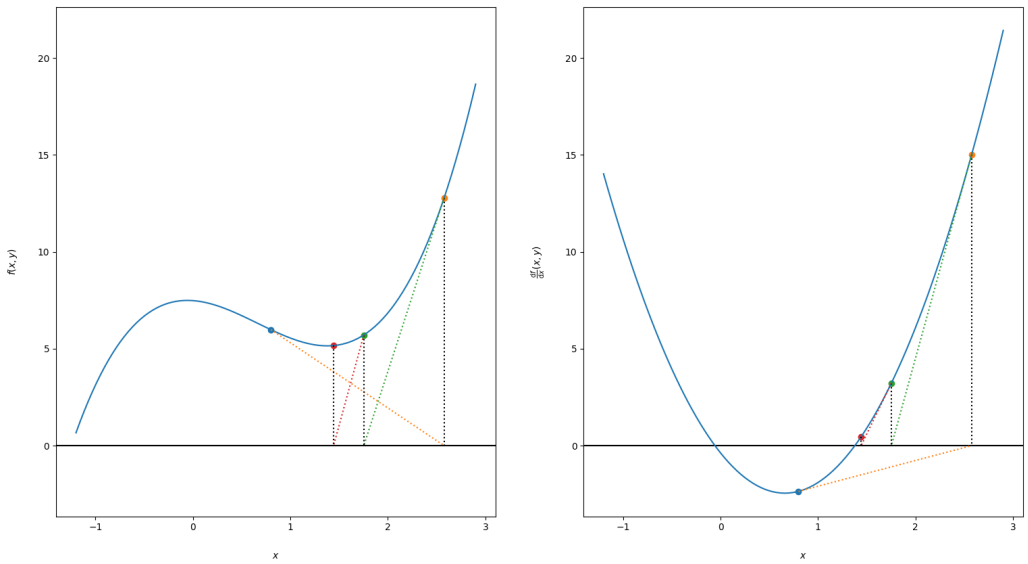


FIGURE 3.2 – À gauche le polynôme de degré trois dont on cherche un minimum local. À droite, sa dérivée. Ce graphique montre qu'il faut se méfier de l'intuition « la méthode de Newton consiste à suivre la tangente au graphe de la fonction ». Lorsqu'on l'applique à un problème de minimisation, elle suit la tangente au graphe *de la dérivée* de la fonction à minimiser.

Chapitre 4

La méthode de Newton en plusieurs variables

Elle a deux applications différentes mais liées :

1. résoudre un système d'équations,
2. minimiser un objectif.

Avant de résoudre un système d'équations, on commence par généraliser la notion de développement limité en plusieurs variables.

4.1 Fonctions de plusieurs variables

On s'intéresse à des fonctions $F : \mathbb{R}^m \to \mathbb{R}^p$ c'est-à-dire à des vecteurs

$$F(x) = \begin{pmatrix} f_1(x_1, x_2, \ldots, x_m) \\ f_2(x_1, x_2, \ldots, x_m) \\ \vdots \\ f_p(x_1, x_2, \ldots, x_m) \end{pmatrix}. \tag{4.1}$$

de p fonctions f_i de m variables x_j. On supposera ces fonctions f_i suffisamment différentiables à chaque fois que ce sera nécessaire. Les m variables forment un vecteur $x = \begin{pmatrix} x_1 & x_2 & \cdots & x_m \end{pmatrix}^T$. Dans l'exemple suivant, où $m = p = 2$, pour éviter les indices, on note f et g au lieu de f_1 et f_2 ; et a et b au lieu de x_1 et x_2.

$$F(x) = \begin{pmatrix} f(a,b) \\ g(a,b) \end{pmatrix} = \begin{pmatrix} a^2 + 2\,a\,b - 1 \\ a^2\,b^2 - b - 1 \end{pmatrix}. \tag{4.2}$$

Voir le fichier `4-graphe_R2_dans_R2.py`.

4.2 Développement limité

La notion de développement limité se généralise aux fonctions de plusieurs variables. Soit par exemple $(a^*, b^*) \in \mathbb{R}^2$ un point. Le développement limité à l'ordre deux des fonctions f et g du

système (4.2) au voisinage de (a^*, b^*) vaut [1]

$$\begin{aligned} f(a^*+k, b^*+\ell) &= f(a^*,b^*) + \tfrac{\partial f}{\partial a}(a^*,b^*)\,k + \tfrac{\partial f}{\partial b}(a^*,b^*)\,\ell + \\ &\qquad \tfrac{1}{2}\,\tfrac{\partial^2 f}{\partial a^2}(a^*,b^*)\,k^2 + \tfrac{\partial^2 f}{\partial a\,\partial b}(a^*,b^*)\,k\,\ell + \tfrac{1}{2}\,\tfrac{\partial^2 f}{\partial b^2}(a^*,b^*)\,\ell^2 + \cdots \\ g(a^*+k, b^*+\ell) &= g(a^*,b^*) + \tfrac{\partial g}{\partial a}(a^*,b^*)\,k + \tfrac{\partial g}{\partial b}(a^*,b^*)\,\ell + \\ &\qquad \tfrac{1}{2}\,\tfrac{\partial^2 g}{\partial a^2}(a^*,b^*)\,k^2 + \tfrac{\partial^2 g}{\partial a\,\partial b}(a^*,b^*)\,k\,\ell + \tfrac{1}{2}\,\tfrac{\partial^2 g}{\partial b^2}(a^*,b^*)\,\ell^2 + \cdots \end{aligned} \tag{4.3}$$

La notation $\partial f/\partial a$ désigne la dérivée partielle de f par rapport à a. C'est une fonction de a et de b aussi. Elle s'obtient en dérivant f de la façon habituelle par rapport à la variable a tout en considérant que b est une constante. De même $\partial f/\partial b$ s'obtient en dérivant f par rapport à b tout en considérant que a est une constante. La notation $\partial^2 f/\partial a^2$ désigne la dérivée partielle seconde obtenue en appliquant $\partial/\partial a$ à $\partial f/\partial a$. La notation $\partial^2 f/(\partial a\,\partial b)$ désigne la dérivée partielle seconde obtenue en appliquant $\partial/\partial a$ à $\partial f/\partial b$. Dans le cas des fonctions considérées dans ce cours, les dérivations partielles commutent, c'est-à-dire qu'on obtient le même résultat quel que soit l'ordre dans lequel on enchaîne les dérivations partielles par rapport à a et à b. Sur l'exemple (4.2) on obtient les expressions suivantes qui sont toutes des fonctions de a et de b

$$\frac{\partial f}{\partial a} = 2\,a + 2\,b\,, \quad \frac{\partial f}{\partial b} = 2\,a\,, \quad \frac{\partial^2 f}{\partial a^2} = 2\,, \quad \frac{\partial^2 f}{\partial a\,\partial b} = 2\,, \quad \frac{\partial^2 f}{\partial b^2} = 0\,,$$

$$\frac{\partial g}{\partial a} = 2\,a\,b^2\,, \quad \frac{\partial g}{\partial b} = 2\,a^2\,b - 1\,, \quad \frac{\partial^2 g}{\partial a^2} = 2\,b^2\,, \quad \frac{\partial^2 g}{\partial a\,\partial b} = 4\,a\,b\,, \quad \frac{\partial^2 g}{\partial b^2} = 2\,a^2\,.$$

En substituant ces valeurs dans la formule générale (4.3) on obtient :

$$\begin{aligned} f(a^*+k, b^*+\ell) &= ((a^*)^2 + 2\,a^*\,b^* - 1) + (2\,a^* + 2\,b^*)\,k + 2\,a^*\,\ell + k^2 + 2\,k\,\ell + \cdots \\ g(a^*+k, b^*+\ell) &= ((a^*)^2\,(b^*)^2 - b^* - 1) + 2\,a^*\,(b^*)^2\,k + (2\,(a^*)^2\,b^* - 1)\,\ell + \\ &\qquad b^*\,k^2 + 4\,a^*\,b^*\,k\,\ell + (a^*)^2\,\ell^2 + \cdots \end{aligned} \tag{4.4}$$

4.3 Écriture matricielle du développement limité

On introduit deux objets importants : le vecteur gradient d'une fonction $f : \mathbb{R}^m \to \mathbb{R}$ et la matrice jacobienne d'une fonction $F : \mathbb{R}^m \to \mathbb{R}^p$.

Définition 5 *(vecteur gradient)*
Soit $f(x_1, x_2, \ldots, x_m)$ une fonction $\mathbb{R}^m \to \mathbb{R}$. Le vecteur gradient *de f, noté ∇f, est le vecteur colonne formé des dérivées partielles (premières) de f.*

1. D'une façon générale, le développement limité d'une fonction de m variables s'écrit :

$$\begin{aligned} f(x_1^*+h_1, x_2^*+h_2, \ldots, x_m^*+h_m) &= \\ \sum \frac{1}{k_1!\,k_2!\cdots k_m!}\,\frac{\partial^{k_1+k_2+\cdots+k_m} f}{\partial x_1^{k_1}\,\partial x_2^{k_2}\cdots\partial x_m^{k_m}}(x_1^*, x_2^*, \ldots, x_m^*)\,h_1^{k_1}\,h_2^{k_2}\cdots h_m^{k_m}\,. \end{aligned}$$

Comme les coordonnées de ∇f sont des fonctions de m variables réelles, ce vecteur lui-même peut être vu comme une fonction $\mathbb{R}^m \to \mathbb{R}^m$.

$$\nabla f(x_1, x_2, \ldots, x_m) \quad = \quad \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_m} \end{pmatrix} \tag{4.5}$$

Revenons à l'exemple (4.2). Chacune des deux fonctions $\mathbb{R}^2 \to \mathbb{R}$ (on a donc $m = 2$) admet un gradient. Ces gradients sont des fonctions $\mathbb{R}^2 \to \mathbb{R}^2$.

$$\nabla f(a, b) \quad = \quad \begin{pmatrix} 2\,a + 2\,b \\ 2\,a \end{pmatrix} \ , \qquad \nabla g(a, b) \quad = \quad \begin{pmatrix} 2\,a\,b^2 \\ 2\,a^2\,b - 1 \end{pmatrix} \ .$$

Définition 6 *(matrice jacobienne)*

Considérons maintenant le cas (4.1) *général d'une fonction $F : \mathbb{R}^m \to \mathbb{R}^p$, formée de p fonctions $f_i(x_1, x_2, \ldots, x_m)$. On définit la* matrice jacobienne *de F, notée J_F, comme la matrice $p \times m$ des dérivées partielles (premières) de F. Les lignes de J_F sont les vecteurs gradients des fonctions $f_i(x_1, x_2, \ldots, x_m)$, écrits en ligne.*

Dit autrement, les lignes de J_F sont les transposées $(\nabla f_1)^T, (\nabla f_2)^T, \ldots, (\nabla f_p)^T$ des vecteurs gradients $\nabla f_1, \nabla f_2, \ldots, \nabla f_p$. La matrice jacobienne de F est donc une fonction $J_F : \mathbb{R}^m \to \mathbb{R}^{p \times m}$.

$$J_F(x_1, x_2, \ldots, x_m) \quad = \quad \begin{pmatrix} (\nabla f_1)^T \\ (\nabla f_2)^T \\ \vdots \\ (\nabla f_p)^T \end{pmatrix} \quad = \quad \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_m} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_m} \\ \vdots & \vdots & & \vdots \\ \frac{\partial f_p}{\partial x_1} & \frac{\partial f_p}{\partial x_2} & \cdots & \frac{\partial f_p}{\partial x_m} \end{pmatrix} .$$

Dans le cas du système (4.2) on a

$$J_F(a, b) \quad = \quad \begin{pmatrix} 2\,a + 2\,b & 2\,a \\ 2\,a\,b^2 & 2\,a^2\,b - 1 \end{pmatrix} .$$

Ces définitions étant introduites, on peut écrire de façon matricielle le développement limité à l'ordre un du système général (4.1). Soient $x^* = \begin{pmatrix} x_1^* & x_2^* & \cdots & x_m^* \end{pmatrix}^T$ et $h = \begin{pmatrix} h_1 & h_2 & \cdots & h_m \end{pmatrix}^T$ deux vecteurs de $\mathbb{R}^m$. Alors

$$F(x^* + h) \quad = \quad F(x^*) + J_F(x^*)\,h + \cdots \tag{4.6}$$

L'expression $J_F(x^*)$ désigne la matrice jacobienne de F, évaluée en le vecteur x^*. Donc $J_F(x^*)$ est une matrice de dimension $p \times m$. L'expression $J_F(x^*)\,h$ désigne le produit scalaire de $J_F(x^*)$ par le vecteur h. Comme les coordonnées du vecteur h représentent m petits déplacements par rapport aux m variables, il est normal que h soit de dimension m et donc que J_F ait m colonnes.

4.4 Exemple

Revenons au cas du système (4.2). La formule (4.6) devient :

$$\begin{pmatrix} f(a^*+k, b^*+\ell) \\ g(a^*+k, b^*+\ell) \end{pmatrix} = \begin{pmatrix} f(a^*, b^*) \\ g(a^*, b^*) \end{pmatrix} + \begin{pmatrix} \frac{\partial f}{\partial a}(a^*, b^*) & \frac{\partial f}{\partial b}(a^*, b^*) \\ \frac{\partial g}{\partial a}(a^*, b^*) & \frac{\partial g}{\partial b}(a^*, b^*) \end{pmatrix} \cdot \begin{pmatrix} k \\ \ell \end{pmatrix} + \cdots \tag{4.7}$$

ou encore

$$\begin{pmatrix} f(a^*+k, b^*+\ell) \\ g(a^*+k, b^*+\ell) \end{pmatrix} = \begin{pmatrix} (a^*)^2 + 2\,a^*\,b^* - 1 \\ (a^*)^2\,(b^*)^2 - b^* - 1 \end{pmatrix} + \begin{pmatrix} 2\,a^* + 2\,b^* & 2\,a^* \\ 2\,a^*\,(b^*)^2 & 2\,(a^*)^2\,b^* - 1 \end{pmatrix} \cdot \begin{pmatrix} k \\ \ell \end{pmatrix} + \cdots$$

4.5 Méthode de Newton pour résoudre un système carré

Un système carré est un système F tel que $m = p$. D'après la formule (4.6), le développement limité de F en $u_0 = x^*$ s'écrit :

$$F(u_0 + h) = F(u_0) + J_F(u_0)\,h + \cdots$$

On cherche un vecteur h tel que $F(u_0 + h) = 0$. En négligeant les termes cachés par les points de suspension on trouve

$$h = -J_F^{-1}(u_0)\,F(u_0) \quad \text{et donc} \quad u_0 + h = u_0 - J_F^{-1}(u_0)\,F(u_0)\,.$$

Comme le développement en série est tronqué, le vecteur $u_1 = u_0 + h$ trouvé n'est qu'une approximation du zéro α recherché et on recommence le calcul en le prenant comme nouveau point de départ. La formule finale est très proche de (3.1).

Proposition 14 *Pour calculer le vecteur u_{n+1} en fonction du vecteur u_n il suffit de*

$$\begin{array}{lrcl} \textit{résoudre} & -J_F(u_n)\,h & = & F(u_n)\,, \\ \textit{prendre} & u_{n+1} & = & u_n + h\,. \end{array} \tag{4.8}$$

On voit que la matrice jacobienne de F joue le rôle de la dérivée dans le cas d'une variable. Pour des raisons de stabilité numérique, on évite de calculer l'inverse de la jacobienne. À chaque itération, on est donc amené à résoudre un système d'équations linéaires.

Proposition 15 *(vitesse de convergence quadratique)*
La vitesse de convergence de la méthode de Newton est encore quadratique (voir formule (3.3)*, page 16) dans le cas de plusieurs variables, sous certaines hypothèses qu'on ne détaillera pas.*

Les idées sont les mêmes que dans le cas d'une variable. On commence par supposer que la suite définie par (4.8) converge vers le zéro α recherché. Il suffit d'adapter la définition de l'erreur à l'itération n par

$$e_n = \|u_n - \alpha\|_2\,.$$

On se contente ci-dessous de vérifier expérimentalement la vitesse de convergence quadratique, sur l'exemple (4.2).

```
import numpy as np
import math

def F (u) :
    a = u[0]
    b = u[1]
    return np.array ([a**2 + 2*a*b - 1, a**2*b**2 - b - 1])

def J_F (u) :
    a = u[0]
    b = u[1]
    return np.array ([[2*a + 2*b, 2*a],[2*a*b**2, 2*a**2*b - 1]])
```

On entre le zéro α recherché (on triche, mais uniquement pour pouvoir estimer l'erreur faite à chaque itération !). On entre aussi le point de départ `u` $= u_0$, supposé être une approximation connue de α. À chaque itération, on met à jour la valeur du vecteur `u` et on affiche une estimation du nombre bits nuls de la mantisse de l'erreur.

```
alpha = np.array([1.568995999, -0.4658228710], dtype=np.float64)
u = np.array([1,-1], dtype=np.float64)
for n in range (8) :
    log2err = math.log2 (np.linalg.norm (alpha - u, 2))
    print ('u[%d] =' % n, u, '-log2(e[%d]) =' % n, - log2err)
    h = np.linalg.solve (- J_F(u), F(u))
    u = u + h
```

Ce nombre de bits double approximativement à chaque itération dès que `u` est suffisamment proche de α, ce qui suggère bien une vitesse de convergence quadratique.

```
u[0] = [ 1. -1.] -log2(e[0]) = 0.3576225387687623
u[1] = [2. 0.] -log2(e[1]) = 0.6560120884639021
u[2] = [ 2.25 -1.  ] -log2(e[2]) = 0.20837366378672414
u[3] = [ 1.87727568 -0.69570871] -log2(e[3]) = 1.3787317272176745
u[4] = [ 1.64896988 -0.52859759] -log2(e[4]) = 3.2980551571474965
u[5] = [ 1.57582906 -0.4715707 ] -log2(e[5]) = 6.807274574222561
u[6] = [ 1.56905332 -0.46587315] -log2(e[6]) = 13.678948567646472
u[7] = [ 1.568996   -0.46582288] -log2(e[7]) = 27.311410597863954
```

Chapitre 5

Optimisation non contrainte

On montre comment appliquer la méthode de Newton et son concurrent, l'algorithme du gradient, pour minimiser un objectif, qui est une fonction à image dans $\mathbb{R}$.

Définition 7 *(objectif)*
Dans un problème d'optimisation, on appelle objectif *la fonction $f : \mathbb{R}^m \to \mathbb{R}$ à minimiser.*

La fonction suivante fournit un exemple d'objectif ($m = 2$) :

$$f(a, b) \quad = \quad a^3 + 2\,a^2 - 2\,a\,b + b^2 + a - 2\,b + 5\,. \tag{5.1}$$

5.1 Jacobienne, gradient et hessienne d'un objectif

Proposition 16 *(gradient et jacobienne d'un objectif)*
Comme un objectif est à image dans $\mathbb{R}$, sa matrice jacobienne ne comporte qu'une seule ligne. Elle est égale à la transposée de son vecteur gradient : $J_f = (\nabla f)^T$.

Voici la matrice jacobienne de l'exemple (5.1). On peut vérifier que les points de coordonnées $(a^*, b^*) = (\frac{1}{3}, \frac{4}{3})$ et $(a^*, b^*) = (-1, 0)$ sont stationnaires (définition ci-dessous).

$$J_f(a, b) \quad = \quad \begin{pmatrix} 3\,a^2 + 4\,a - 2\,b + 1 & -2\,a + 2\,b - 2 \end{pmatrix}\,. \tag{5.2}$$

Définition 8 *(point stationnaire)*
On appelle point stationnaire *d'un objectif tout point $x^* \in \mathbb{R}^m$ qui annule sa matrice jacobienne — ou son gradient.*

> *Necessary conditions* for optimality are derived by assuming that x^* is a local minimizer and then proving facts about $\nabla f(x^*)$ and $\nabla^2 f(x^*)$.
>
> [2, page 15]

Le principe est le même que dans le cas d'une variable : on cherche d'abord les points stationnaires de f, qui correspondent informellement aux points du graphe ayant un « plan tangent » horizontal. Ensuite, pour chacun d'eux, on détermine s'il s'agit d'un minimum local en étudiant la matrice qui joue le rôle de la dérivée seconde et qui est appelée *matrice hessienne* de f.

Définition 9 *(matrice hessienne)*

La matrice hessienne H_f *(parfois notée $\nabla^2 f$) d'un objectif f est définie comme la matrice des dérivées partielles secondes de f. C'est une matrice symétrique (parce qu'on a supposé que les dérivations partielles commutent entre elles). Elle est de dimension $m \times m$.*

Comme les dérivées partielles secondes de f sont des fonctions de $x_1, x_2, \ldots, x_m$, la matrice H_f aussi et on a

$$H_f(x_1, x_2, \ldots, x_m) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_m} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_m} \\ \vdots & \vdots & & \vdots \\ \frac{\partial^2 f}{\partial x_m \partial x_1} & \frac{\partial^2 f}{\partial x_m \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_m^2} \end{pmatrix}. \tag{5.3}$$

Sur l'exemple on a :

$$H_f(a, b) = \begin{pmatrix} 6\,a + 4 & -2 \\ -2 & 2 \end{pmatrix}. \tag{5.4}$$

Cette matrice hessienne joue deux rôles importants, précisés dans les deux sections suivantes.

5.2 Méthode de Newton pour calculer un point stationnaire

Proposition 17 *(la hessienne est la jacobienne du gradient)*

La matrice hessienne H_f d'un objectif f est égale à la matrice jacobienne du gradient (ou de la matrice jacobienne) de f. En d'autres termes,

$$H_f = J_{\nabla f}. \tag{5.5}$$

C'est cette proposition qui justifie l'utilisation de la méthode de Newton (4.8) pour calculer un point stationnaire de f : il suffit de partir d'une approximation $u_0 = x^* = (\ x_1^* \quad x_2^* \quad \cdots \quad x_m^*\ )^T$ de ce point stationnaire et d'appliquer la formule suivante, obtenue en remplaçant f et J_f par ∇f et H_f dans (4.8) (on rappelle que $h = (\ h_1 \quad h_2 \quad \cdots \quad h_m\ )^T$ est un vecteur de dimension m) :

$$\begin{array}{rrcl} \text{résoudre} & -H_f(u_n)\,h & = & \nabla f(u_n)\,, \\ \text{prendre} & u_{n+1} & = & u_n + h\,. \end{array} \tag{5.6}$$

5.3 Exemple

Pour l'animation graphique, voir `5-Newton_sans_controle.py`.

```
a0 = -1/5
b0 = 1
un = np.array ([a0,b0], dtype=np.float64)
for i in range (6) :
```

```
a = un[0]
b = un[1]
print ('u[%d] =' % i, un, 'f(u[%d]) =' % i, f(a,b))
H_f_un = H_f (a,b)          # hessienne de f en u[n]
nabla_f_un = nabla_f (a,b)  # gradient de f en u[n]
h = np.linalg.solve (- H_f_un, nabla_f_un)
un = un + h
```

On part du point $u_0 = (-\frac{1}{5}, 1)$. On voit que la suite converge vers le point stationnaire $(\frac{1}{3}, \frac{4}{3})$. À chaque itération, on donne l'altitude $f(u_n)$ du point u_n On remarque un phénomène non intuitif : l'altitude du point u_1 est supérieure à celle de u_0.

```
u[0] = [-0.2  1. ] f(u[0]) = 4.272
u[1] = [1.4 2.4] f(u[1]) = 7.304000000000004
u[2] = [0.66153846 1.66153846] f(u[2]) = 4.065605826126537
u[3] = [0.38747026 1.38747026] f(u[3]) = 3.8208350937294266
u[4] = [0.33536635 1.33536635] f(u[4]) = 3.8148230895060893
u[5] = [0.33333642 1.33333642] f(u[5]) = 3.8148148148339165
```

5.4 Un point stationnaire est un minimum local si la hessienne est définie positive

Proposition 18 *(condition de minimum local)*
Soit $x^* = (\ x_1^* \quad x_2^* \quad \cdots \quad x_m^*\ )^T$ un point de $\mathbb{R}^m$ qui annule le gradient ∇f (ou la matrice jacobienne J_f) de l'objectif f. Alors x^* est un minimum local de f si la matrice hessienne $H_f(x^*)$ — qui est symétrique — est définie positive.

Pour nous en convaincre, rappelons (formule (4.4)) le développement limité à l'ordre deux d'une fonction f de deux variables :

$$f(a^* + k, b^* + \ell) = f(a^*, b^*) + \frac{\partial f}{\partial a}(a^*, b^*)\, k + \frac{\partial f}{\partial b}(a^*, b^*)\, \ell + \tag{5.7}$$

$$\frac{1}{2}\frac{\partial^2 f}{\partial a^2}(a^*, b^*)\, k^2 + \frac{\partial^2 f}{\partial a\, \partial b}(a^*, b^*)\, k\, \ell + \frac{1}{2}\frac{\partial^2 f}{\partial b^2}(a^*, b^*)\, \ell^2 + \cdots \tag{5.8}$$

Le point clé, c'est que les termes d'ordre deux (5.8) s'obtiennent par l'expression suivante qui fait intervenir la matrice hessienne de f

$$\frac{1}{2}\frac{\partial^2 f}{\partial a^2}(a^*, b^*)\, k^2 + \frac{\partial^2 f}{\partial a\, \partial b}(a^*, b^*)\, k\, \ell + \frac{1}{2}\frac{\partial^2 f}{\partial b^2}(a^*, b^*)\, \ell^2 = \frac{1}{2}\begin{pmatrix} k & \ell \end{pmatrix} H_f(a^*, b^*) \begin{pmatrix} k \\ \ell \end{pmatrix}.$$

Supposons que le point $(\ a^* \quad b^*\ )^T$ soit l'abscisse d'un point stationnaire. Il annule le vecteur gradient de f et donc les deux dérivées partielles premières de f (qui apparaissent en (5.7)). Notons $h = (\ k \quad \ell\ )^T$. On a alors :

$$f(a^* + k, b^* + \ell) = f(a^*, b^*) + \underbrace{(\nabla f(x^*))^T\, h}_{=0} + h^T\, H_f(a^*, b^*)\, h + \cdots \tag{5.9}$$

Supposons la matrice $H_f(a^*, b^*)$ définie positive. Alors, quel que soit le vecteur h non nul, on a $h^T\, H_f(a^*, b^*)\, h > 0$. Si on suppose de plus que k et ℓ sont petits, les termes cachés dans les points

de suspension sont négligeables. Par conséquent, quelle que soit la direction h suivant laquelle on s'éloigne du point d'abscisse (a^*, b^*), le graphe de l'objectif monte : on a bien affaire à un minimum local.

5.5 Minima locaux, maxima locaux, cols

Les valeurs propres d'une matrice symétrique sont toujours réelles. Une matrice symétrique est définie positive si et seulement si toutes ses valeurs propres sont strictement positives.

D'après la proposition 18, un point stationnaire x^* est un minimum local si et seulement si toutes les valeurs propres de la matrice hessienne $H_f(x^*)$ sont strictement positives.

En adaptant un peu le raisonnement tenu dans la section précédente, on voit que x^* est un *maximum local* si et seulement si toutes les valeurs propres de la matrice hessienne $H_f(x^*)$ sont strictement négatives.

Si la matrice hessienne $H_f(x^*)$ a des valeurs propres de signes différents, on dit que le point stationnaire x^* est un *col*. Le graphe de l'objectif monte ou descend en fonction de la direction h. Il est assez facile de s'en convaincre en supposant que h est un vecteur propre de la hessienne $H_f(x^*)$ de valeur propre λ. Dans ce cas, on a $h^T\, H_f(x^*)\, h = \lambda\, h^T\, h$. Le produit scalaire $h^T\, h$ est une somme de carrés et donc un nombre positif. La contribution des termes d'ordre deux dans le développement limité $f(x^* + h)$ est donc du même signe que la valeur propre λ. Le graphe de f monte dans la direction h si $\lambda > 0$. Il descend si $\lambda < 0$.

5.6 Exemple

On vérifie que le point obtenu au cours de la simulation précédente est bien un minimum local : le gradient est nul et la hessienne est définie positive (la factorisation de Cholesky réussit).

```
>>> nabla_f_un
array([1.23617641e-05, 0.00000000e+00])
>>> import scipy.linalg as nla
>>> nla.cholesky (H_f_un, lower=True)
array([[ 2.44949353,  0.        ],
       [-0.81649532,  1.15470143]])
```

5.7 Nécessité de contrôler la longueur du pas

Lorsqu'on calcule une suite de vecteurs (u_n) de $\mathbb{R}^m$ par la méthode de Newton (5.11), pour minimiser un objectif f, il peut arriver, comme le montre une simulation précédente, que les altitudes des points (les valeurs des $f(u_n)$) augmentent, parce que le pas h suivi est trop long. Ce phénomène peut paraître suprenant mais on l'a déjà rencontré. Voir figure 3.2. Dans un problème de minimisation d'un objectif, il est donc essentiel de contrôler la longueur du pas, même si on est proche du minimum local. Avant d'aborder ce point, on présente l'algorithme du gradient.

5.8 Le gradient donne la direction de la plus grande pente

Soient $x^* = (\ x_1^* \quad x_2^* \quad \cdots \quad x_m^*\ )^T$ un point de $\mathbb{R}^m$ et $h = (\ h_1 \quad h_2 \quad \cdots \quad h_m\ )^T$ une direction. La dérivée directionnelle se définit classiquement comme une limite. Dans ce cours, on adopte la

définition suivante :

Définition 10 *(dérivée directionnelle)*
On appelle dérivée directionnelle *de f dans la direction h en x^* le scalaire*

$$D_h(f)(x^*) \quad = \quad h^T \nabla f(x^*)\,.$$

Le fichier `6-derivee_directionnelle.py` fournit une interprétation graphique de la dérivée directionnelle : à partir d'un point $u_0 \in \mathbb{R}^2$ on calcule une direction h ; on intersecte le graphe $z = f(a,b)$ avec le plan vertical qui contient les points u_0 et $u_0 + h$; on obtient une courbe $\mathscr{C}$ qui appartient au graphe ; si h est de longueur 1, la dérivée directionnelle $D_h(f)(u_0)$ est égale à la pente de la tangente à la courbe $\mathscr{C}$ en $(u_0, f(u_0))$. Si h n'est pas de longueur 1 alors la dérivée directionnelle $D_h(f)(u_0)$ est égale à la pente de la tangente à la courbe $\mathscr{C}$ en $(u_0, f(u_0))$ multipliée par $\|h\|_2$.

Proposition 19 *(direction d'une descente)*
Le graphe de f descend dans la direction h si la dérivée directionnelle $D_h(f)(x^*)$ est négative :

$$h^T \nabla f(x^*) \quad < \quad 0\,.$$

Le signe de la dérivée directionnelle indique donc si le graphe monte ou descend. Fixons h à la longueur 1 pour simplifier. Alors la valeur absolue de la dérivée directionnelle donne l'intensité de la montée ou de la descente. Comme le produit scalaire $h^T \nabla f(x^*)$ est maximal en valeur absolue lorsque h est parallèle au gradient $\nabla f(x^*)$ (inégalité de Cauchy-Schwartz) on conclut que

Proposition 20 *(plus grande pente)*
Le vecteur $h = -\nabla f(x^*)$ donne la direction de la plus grande pente descendante.

5.9 Algorithme du gradient

Il consiste à calculer une suite de points (u_n) qui s'obtiennent en suivant la plus grande pente à chaque itération. Les scalaires α_n contrôlent la longueur du pas. Ils sont recalculés à chaque itération :

$$\begin{array}{lrcl} \text{prendre} & h & = & -\nabla f(u_n)\,, \\ \text{déterminer} & \alpha_n & & \text{à partir de } h \text{ et de } u_n \\ \text{prendre} & u_{n+1} & = & u_n + \alpha_n\, h\,. \end{array} \tag{5.10}$$

5.10 Méthode de Newton avec contrôle du pas

Ici encore, les scalaires α_n sont recalculés à chaque itération. Attention : sans contrôle du pas, la méthode de Newton peut être appliquée à la recherche d'un point stationnaire quelconque ; mais l'algorithme de contrôle du pas introduit dans la section suivante est conçu pour une recherche de minimum local :

$$\begin{array}{lrcl} \text{résoudre} & -H_f(u_n)\, h & = & \nabla f(u_n)\,, \\ \text{déterminer} & \alpha_n & & \text{à partir de } h \text{ et de } u_n \\ \text{prendre} & u_{n+1} & = & u_n + \alpha_n\, h\,. \end{array} \tag{5.11}$$

5.11 Contrôle de la longueur du pas

On cherche une stratégie qui contrôle la longueur du pas qui soit adaptée à l'algorithme du gradient mais aussi à la méthode de Newton dans le contexte d'une recherche de minimum local. L'algorithme ci-dessous est un des plus simples possibles. Il est appelé *backtrack line search* :

$$
\begin{array}{lll}
\text{paramètres} & u = u_n, h & \\
\text{résultat} & \alpha = \alpha_n & \text{(la longueur du pas)} \\
\text{fixer} & 0 < \varrho, c < 1 & \text{(deux constantes)} \\
\text{initialiser} & 0 < \alpha \leqslant 1 & \text{(prendre 1 pour Newton)} \\
\text{tant que} & f(u + \alpha\, h) > f(u) + c\,\alpha\, h^T\, \nabla f(u) & \text{faire} \\
 & \quad \alpha \leftarrow \varrho\, \alpha & \\
\text{fin} & & \\
\text{retourner} & \alpha &
\end{array}
\tag{5.12}
$$

Le flottant α contrôle la longueur du pas. L'algorithme réduit α en le multipliant par ϱ à chaque itération. Dans le test de la boucle l'expression $f(u + \alpha\, h)$ donne la valeur de l'objectif si on se déplace de u en suivant le vecteur $\alpha\, h$. L'expression $f(u) + c\,\alpha\, h^T\, \nabla f(u)$ donne ce qu'on obtiendrait si on accroissait l'objectif $f(u)$ en suivant une droite dont le vecteur directeur serait donné par la dérivée directionnelle $D_h(f)(u)$, tout en faisant un pas de longueur contrôlée par c. Voir la figure 5.1.

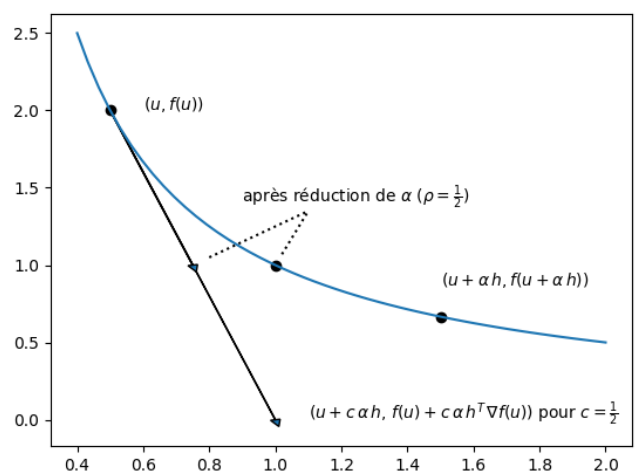


FIGURE 5.1 – On a $f(u + \alpha\, h) > f(u) + c\,\alpha\, h^T\, \nabla f(u)$. L'algorithme *backtrack line search* réduit α. À l'itération suivante la nouvelle valeur de α est acceptée.

5.12 Exemple

Pour une animation graphique, voir `5-Newton_et_gradient_avec_controle.py`. Les points bleus sont calculés avec la méthode de Newton. Les points rouges sont calculés par l'algorithme du gradient (les gradients sont représentés).

Dans la simulation suivante, on a appliqué la méthode de Newton sur l'exemple précédent en lui appliquant l'algorithme de contrôle du pas ($\varrho = c = \frac{1}{2}$ et valeur initiale de $\alpha_n = 1$). On remarque que l'altitude des points diminue à chaque itération. Pour autant, la vitesse de convergence est moins élevée (le contrôle du pas rend l'algorithme plus fiable mais casse un peu sa vitesse de convergence quadratique).

```
u[0] = [-0.2  1. ] f(u[0]) = 4.272
alpha = 0.250000
u[1] = [0.2  1.35] f(u[1]) = 3.8705
alpha = 0.500000
u[2] = [0.275 1.35 ] f(u[2]) = 3.8270468749999997
alpha = 0.500000
u[3] = [0.30556507 1.34306507] f(u[3]) = 3.8177418064219246
alpha = 0.500000
u[4] = [0.31975092 1.33850092] f(u[4]) = 3.815532835399965
alpha = 0.500000
u[5] = [0.32661275 1.33598775] f(u[5]) = 3.8149927344529067
alpha = 0.500000
```

Dans cette simulation-ci, on a appliqué l'algorithme du gradient sur l'exemple précédent en lui appliquant l'algorithme de contrôle du pas ($\varrho = c = \frac{1}{2}$ et valeur initiale de $\alpha_n = \frac{3}{4}$). On observe que la vitesse de convergence est moins élevée que celle de la méthode de Newton, même avec contrôle du pas.

```
u[0] = [-0.2  1. ]
alpha = 0.750000
u[1] = [-0.1949339   0.99803485]
alpha = 0.750000
u[2] = [-0.18986779  0.99606971]
alpha = 0.750000
u[3] = [-0.18480169  0.99410456]
alpha = 0.750000
u[4] = [-0.17973558  0.99213942]
alpha = 0.750000
u[5] = [-0.17466948  0.99017427]
alpha = 0.750000
```

5.13 Discussion

L'algorithme du gradient est plus intuitif que la méthode de Newton. Informatiquement, il est aussi beaucoup plus simple à mettre en œuvre puisqu'il n'utilise que la dérivée première de la fonction f (le gradient). Il est donc normal qu'il soit particulièrement adapté à des problèmes où le nombre de variables est très grand, comme les réseaux de neurones (un petit réseau de neurones peut comporter une fonction f à $m = 10000$ variables). Malheureusement, il peut être considérablement plus lent que la méthode de Newton lorsque les deux méthodes s'appliquent.

Une raison simple permet de comprendre pourquoi la méthode de Newton est plus rapide : sa formule provient d'une meilleure approximation de la fonction f (le développement limité est tronqué à l'ordre deux) ; alors que l'algorithme du gradient provient d'une approximation à l'ordre un de la fonction.

Dans les deux cas (méthode de Newton et algorithme du gradient), une direction h est d'abord choisie et il reste à contrôler la longueur du pas : les méthodes présentées en cours appliquent la stratégie de *recherche linéaire* (en Anglais, *line search methods*) mais d'autres stratégies comme la recherche de *régions de confiance* (en Anglais, *trust region methods*) sont possibles.

> In a sense, the line search and trust region approaches differ in the order in which they choose the *direction* and *distance* of the move to the next iterate. Line search starts by fixing the direction and then identifying an appropriate distance, namely the step length α. In trust region, we first choose a maximum distance — the trust region radius — and then seek a direction and step that attain the best improvement possible subject to this distance constraint.
>
> [2, page 20]

Enfin, la méthode de Newton sans contrôle du pas permet de calculer des points stationnaires qui ne sont pas des minima locaux. Une application est introduite dans le chapitre suivant.

Chapitre 6

Optimisation sous contrainte

6.1 Le gradient est orthogonal aux courbes de niveau

Pour pouvoir parler d'orthogonalité il faut avoir affaire à des vecteurs. Pour cette raison, on est amené à introduire la définition d'espace (vectoriel) tangent. C'est une notion de géométrie différentielle dont la définition classique est compliquée. Dans ce cours, on se contentera de la définition suivante.

Définition 11 *(espace tangent)*
Soient $g : \mathbb{R}^m \to \mathbb{R}$ une fonction, $\mathscr{S}$ l'ensemble des points de $\mathbb{R}^m$ solutions de l'équation $g(x) = 0$ et x^* un point de $\mathscr{S}$. On appelle espace tangent *à $\mathscr{S}$ en x^* l'espace vectoriel $\mathscr{T}$ des vecteurs $x \in \mathbb{R}^m$ orthogonaux au gradient $\nabla g(x^*)$ de g, en x^* :*

$$\mathscr{T} \underset{\text{def}}{=} \{x \in \mathbb{R}^m \mid x^T \nabla g(x^*) = 0\}.$$

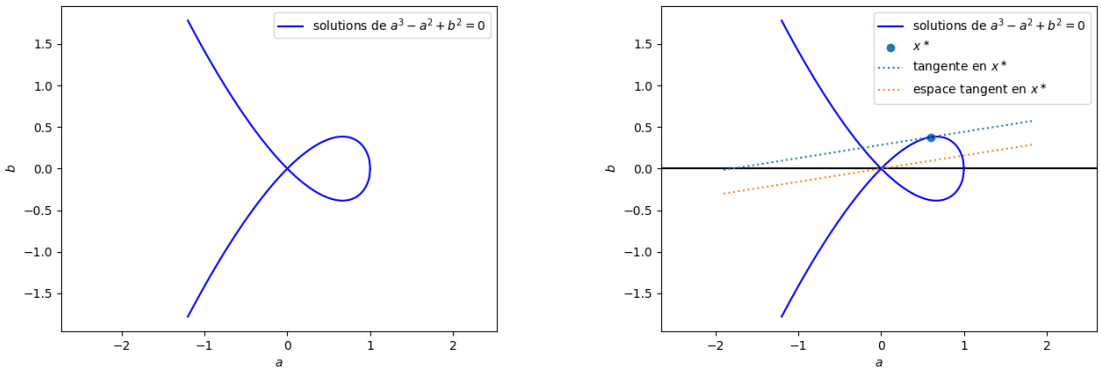


FIGURE 6.1 – L'ensemble $\mathscr{S}$ des solutions d'une équation implicite peut présenter des points d'autointersection où l'espace tangent n'est pas défini (à gauche). Les autres points de la courbe admettent chacun un espace tangent qu'on peut visualiser comme des droites passant par l'origine. Chacune de ces droites est parallèle à la tangente correspondante (à droite).

Exemple dans le cas $m = 2$. Prenons $g(a, b) = a^3 - a^2 + b^2$. L'ensemble $\mathscr{S}$ est une courbe plane (voir figure 6.1). Certains points de $\mathscr{S}$ peuvent ne pas comporter d'espace tangent. C'est le cas du point $x^* = \begin{pmatrix} 0 & 0 \end{pmatrix}^T$ sur lequel les deux dérivées partielles s'annulent. On supposera par la suite que l'espace tangent existe en chacun des points considérés. C'est le cas si les m dérivées partielles de g sont non nulles en x^*.

Pour un point $x^* \in \mathbb{R}^2$ donné, l'espace tangent $\mathscr{T}$ est une droite passant par l'origine. Cette droite est parallèle à la tangente à la courbe en x^*. L'espace tangent et la tangente ne désignent donc pas la même chose. Toutefois, un vecteur est orthogonal (respectivement parallèle) à l'un si et seulement si il est orthogonal (respectivement parallèle) à l'autre. Pour ce type de question, on peut donc se permettre de les confondre.

Calculatoirement, sur l'exemple, on définit l'espace tangent en un point $\begin{pmatrix} a^* & b^* \end{pmatrix}^T$ de la courbe ainsi :

$$\mathscr{T} \quad = \quad \{\begin{pmatrix} a & b \end{pmatrix}^T \mid (3\,(a^*)^2 - 2\,a^*)\,a + 2\,b^*\,b = 0\}\,.$$

On remarque qu'à un renommage près des variables, on peut s'appuyer sur la notion de *différentielle*. Sur l'exemple on a :

$$\mathrm{d}g \quad = \quad \frac{\partial g}{\partial a}\,\mathrm{d}a + \frac{\partial g}{\partial b}\,\mathrm{d}b \quad = \quad (3\,a^2 - 2\,a)\,\mathrm{d}a + 2\,b\,\mathrm{d}b\,.$$

L'espace tangent $\mathscr{T}$ s'obtient en substituant à a et à b les coordonnées a^* et b^* d'un point de la courbe dans la différentielle de g et en résolvant l'équation linéaire (les inconnues étant $\mathrm{d}a$ et $\mathrm{d}b$).

Le gradient est orthogonal aux courbes de niveau. Prenons le cas d'un objectif $f(x) = f(a, b)$ à minimiser. Une courbe de niveau est définie par une équation $f(x) = c$ où c est une constante quelconque. En posant $g(x) = f(x) - c$ on voit que l'ensemble $\mathscr{S}$ est la courbe de niveau. En un point x^* quelconque de cette courbe, le gradient $\nabla g(x^*) = \nabla f(x^*)$ de la fonction f est donc orthogonal à la tangente à la courbe de niveau (et donc à la courbe elle-même) en x^*.

Exemple dans le cas $m = 3$. Prenons le cas d'un objectif $f(a, b)$ à minimiser et $g(x) = g(a, b, z) = z - f(a, b)$. L'ensemble $\mathscr{S}$ est le graphe de l'objectif. Prenons un point du graphe $x^* = \begin{pmatrix} a^* & b^* & f(a^*, b^*) \end{pmatrix}^T$. Le gradient $\nabla g(x^*) = \begin{pmatrix} -\frac{\partial f}{\partial a}(x^*) & -\frac{\partial f}{\partial b}(x^*) & 1 \end{pmatrix}^T$ est orthogonal à l'espace tangent au graphe en x^*, qu'on peut visualiser comme un plan passant par l'origine, parallèle au plan tangent au graphe en x^*.

6.2 Multiplicateurs de Lagrange

La méthode des *multiplicateurs de Lagrange* permet de transformer un problème d'optimisation sous contrainte en un problème d'optimisation sans contrainte, au prix de l'introduction d'une variable supplémentaire appelée *multiplicateur de Lagrange*.

Pour simplifier, on se place dans le cas le plus simple : celui d'une unique contrainte d'égalité $c(x) = 0$. Le problème (6.4) devient

$$\min_{x \in \mathbb{R}^m} f(x) \quad \text{sous la contrainte} \quad c(x) = 0\,. \tag{6.1}$$

Proposition 21 *(gradients parallèles en une solution optimale)*

Soit $x^* \in \mathbb{R}^m$ une solution optimale de (6.1)*. Alors le vecteur gradient de l'objectif ∇f et celui de la contrainte ∇c sont parallèles en x^*. En d'autres termes, il existe $\lambda \in \mathbb{R}$ (un* multiplicateur de Lagrange*) tel que*

$$\nabla f(x^*) \quad = \quad \lambda \, \nabla c(x^*) \, . \tag{6.2}$$

Preuve. Voir section 6.8. □

6.3 Lagrangien d'un problème d'optimisation sous contrainte

Introduisons le nouvel objectif suivant, appelé *Lagrangien* du problème (6.1), où λ est une variable supplémentaire (le multiplicateur de Lagrange) :

$$\mathscr{L}(x, \lambda) \quad = \quad f(x) \pm \lambda \, c(x) \, . \tag{6.3}$$

Ce Lagrangien dépend des m variables $x_1, x_2, \ldots, x_m$ et de la nouvelle variable λ. Le gradient du Lagrangien est un vecteur de $\mathbb{R}^{m+1}$ qui s'écrit :

$$\nabla \mathscr{L}(x, \lambda) \quad = \quad \begin{pmatrix} \frac{\partial \mathscr{L}}{\partial x_1} \\ \frac{\partial \mathscr{L}}{\partial x_2} \\ \vdots \\ \frac{\partial \mathscr{L}}{\partial x_m} \\ \frac{\partial \mathscr{L}}{\partial \lambda} \end{pmatrix} \quad = \quad \begin{pmatrix} \frac{\partial f}{\partial x_1} - \lambda \, \frac{\partial c}{\partial x_1} \\ \frac{\partial f}{\partial x_2} - \lambda \, \frac{\partial c}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_m} - \lambda \, \frac{\partial c}{\partial x_m} \\ c \end{pmatrix} \quad = \quad \begin{pmatrix} \nabla f - \lambda \, \nabla c \\ c \end{pmatrix} .$$

Par conséquent, les points de $\mathbb{R}^{m+1}$ qui annulent le gradient du Lagrangien (c'est-à-dire les points stationnaires du Lagrangien) sont des points $\begin{pmatrix} x^* & \lambda^* \end{pmatrix}^T$ tels que $c(x^*) = 0$ (la contrainte est satisfaite) et tels que les gradients $\nabla f(x^*)$ et $\nabla c(x^*)$ sont parallèles. Pour déterminer une solution optimale de (6.1), on peut donc commencer par chercher les points stationnaires du Lagrangien (6.3).

6.4 Résolution d'un problème d'optimisation sous contrainte

Un Lagrangien peut prendre une valeur négative arbitrairement grande en valeur absolue puisque le multiplicateur de Lagrange est un réel de signe quelconque. Et donc, quand on passe au Lagrangien, on transforme un problème de recherche d'un minimum local en un problème de recherche d'un point stationnaire qui n'est pas un minimum (c'est un col). La méthode de Newton sans contrôle du pas est adaptée à ce nouveau problème mais le sous-algorithme de contrôle du pas ainsi que l'algorithme du gradient ne le sont pas : ils sont conçus pour une recherche de minimum.

6.5 Exemple

Voir `6-gradients_objectif_et_contrainte_paralleles.py`. On cherche à minimiser l'objectif $f(a, b)$ sous la contrainte que le point (a, b) doit appartenir au cercle de rayon 1, centré sur l'origine. On utilise une méthode de Newton sans contrôle du pas.

```
def f (a,b) :
    return a**3 + 2*a**2 - 2*a*b + b**2 + a - 2*b + 5

def nabla_f (a,b) :
    return np.array ([3*a**2+4*a-2*b+1, -2*a+2*b-2])

def c (a,b) :
    return a**2 + b**2 - 1

def nabla_c (a, b) :
    return np.array ([2*a, 2*b])

def Lagrangien (a,b,lmbda) :
    return f(a,b) + lmbda*c(a,b)

def nabla_Lagrangien (a,b,lmbda) :
    return np.array ([2*lmbda*a+3*a**2+4*a-2*b+1,
                      2*lmbda*b-2*a+2*b-2,
                      a**2+b**2-1])

def Hessian_Lagrangien (a,b,lmbda) :
    return np.array ([[2*lmbda+6*a+4,        -2, 2*a],
                      [           -2, 2*lmbda+2, 2*b],
                      [          2*a,       2*b, 0]])

u = np.array ([0.4, 1.5, 0])
for i in range (6) :
    a = u[0]
    b = u[1]
    lmbda = u[2]
    z = Lagrangien (a, b, lmbda)
    print ('u[%d] =' % i, u, 'f(u[%d]) =' % i, z)
    H = Hessian_Lagrangien(a,b,lmbda)
    g = nabla_Lagrangien(a,b,lmbda)
    h = np.linalg.solve (- H, g)
    u = u + h
    grad_f = nabla_f (a,b)
    grad_c = nabla_c (a,b)
    grad_f = (1/np.linalg.norm(grad_f,2)) * grad_f
    grad_c = (1/np.linalg.norm(grad_c,2)) * grad_c
    print ('    nabla f normalisé =', grad_f)
    print ('    nabla c normalisé =', grad_c)
    print ('    valeur de la contrainte =', c(a,b))
```

À l'exécution, la suite (u_n) converge vers la solution optimale. On vérifie que la contrainte s'annule et que le gradient de l'objectif et celui de la contrainte sont bien parallèles.

```
u[0] = [0.4 1.5 0. ] f(u[0]) = 3.834
    nabla f normalisé = [0.37139068 0.92847669]
    nabla c normalisé = [0.25766265 0.96623494]
    valeur de la contrainte = 1.4100000000000001
u[1] = [0.24001168 1.07266355 0.11156542] f(u[1]) = 3.8826546787340046
```

```
    nabla f normalisé = [-0.0372126  -0.99930737]
    nabla c normalisé = [0.21835376 0.97586968]
    valeur de la contrainte = 0.20821270211918086
u[2] = [0.19430946 0.9858355  0.20338247] f(u[2]) = 3.8962026040829474
    nabla f normalisé = [-0.19107716 -0.98157502]
    nabla c normalisé = [0.19338077 0.98112378]
    valeur de la contrainte = 0.009627803428418069
u[3] = [0.19228005 0.98135243 0.21488311] f(u[3]) = 3.8962961352040977
    nabla f normalisé = [-0.19230936 -0.98133435]
    nabla c normalisé = [0.19227773 0.98134055]
    valeur de la contrainte = 2.421639433247691e-05
u[4] = [0.19227751 0.98134059 0.21494771] f(u[4]) = 3.896296136640436
    nabla f normalisé = [-0.19227751 -0.98134059]
    nabla c normalisé = [0.19227751 0.98134059]
    valeur de la contrainte = 1.4665313408102065e-10
u[5] = [0.19227751 0.98134059 0.21494771] f(u[5]) = 3.896296136640436
    nabla f normalisé = [-0.19227751 -0.98134059]
    nabla c normalisé = [0.19227751 0.98134059]
    valeur de la contrainte = 0.0
```

6.6 Forme générale d'un problème d'optimisation sous contrainte

Plus généralement, la forme générale d'un problème d'optimisation sous contrainte est

$$\min_{x\in\mathbb{R}^m} f(x) \quad \text{sous la contrainte} \quad \left\{ \begin{array}{rcll} c_i(x) & = & 0\,, & i \in \mathscr{E}\,, \\ c_i(x) & \geqslant & 0\,, & i \in \mathscr{I} \end{array} \right. \tag{6.4}$$

où $\mathscr{E}$ et $\mathscr{I}$ sont deux ensembles d'indices. Les contraintes c_i pour $i \in \mathscr{E}$ sont les *contraintes d'égalités*. Les contraintes c_i pour $i \in \mathscr{I}$ sont les *contraintes d'inégalités*.

Le Lagrangien d'un problème d'optimisation général (6.4) est défini par

$$\mathscr{L}(x,\lambda) \quad = \quad f(x) \pm \sum_{i\in\mathscr{E}\cup\mathscr{I}} \lambda_i\, c_i(x)\,.$$

Une contrainte $c(x) \geqslant 0$ est dite *active en* x^* si $c(x^*) = 0$; elle est dite *inactive en* x^* si $c(x^*) > 0$.

Les conditions nécessaires d'optimalité décrites dans le cas d'une unique contrainte se généralisent. Sur le principe en tous cas, les contraintes d'inégalités se gèrent comme des contraintes d'égalités puisqu'elles sont soit actives et elles se comportent alors comme des contraintes d'égalités soit inactives et on peut alors les oublier. Dans le cas d'un problème d'optimisation général (6.4), les conditions suffisantes d'optimalité sont souvent appelées conditions de Karush, Kuhn et Tucker (en Anglais, *KKT conditions*).

6.7 Exemples de problèmes d'optimisation sous contrainte

6.7.1 Régression de ridge et Lasso

De nombreux exemples apparaissent dans [1, section 3.4]. Typiquement, on part [1, chapter 3] d'un problème initial où on dispose de p points expérimentaux (x_i, y_i) où chaque x_i est lui-même un vecteur de $\mathbb{R}^m$ et $y_i \in \mathbb{R}$. On cherche un vecteur $x = (\beta_0, \beta_1, \ldots, \beta_m)$ de paramètres

qui correspondent le mieux aux points expérimentaux (le nombre de variables est donc ici $m+1$). L'objectif du problème initial est une somme de carrés définie comme suit :

$$\begin{aligned} f_{\text{init}}(x) &= r_1^2(x) + r_2^2(x) + \cdots + r_p^2(x)\,, \\ r_i(x) &= \beta_0 + x_{i1}\,\beta_1 + x_{i2}\,\beta_2 + \cdots + x_{im}\,\beta_m - y_i\,. \end{aligned} \tag{6.5}$$

Si on s'arrête-là, on a affaire à un simple problème de moindres carrés linéaires.

> Shrinkage methods are more continuous than best-subset selection and don't suffer as much from high variability. Ridge regression shrinks the regression coefficients by imposing a penalty on their size.
>
> [1, section 3.4]

L'objectif de la *régression de ridge* s'écrit en ajoutant une pénalité à l'objectif f_{init} :

$$f_{\text{ridge}}(x) = f_{\text{init}}(x) + \lambda\,(\beta_1^2 + \beta_2^2 + \cdots + \beta_m^2)\,.$$

Le paramètre λ est un multiplicateur de Lagrange. Le problème ci-dessus est équivalent au problème suivant, où S est un autre paramètre (à toute valeur de λ correspond une valeur de S et réciproquement) :

$$\min f_{\text{init}}(x) \quad \text{sous la contrainte} \quad \beta_1^2 + \beta_2^2 + \cdots + \beta_m^2 \leqslant S\,.$$

Notons X la matrice X, de dimension $p \times m$ qui contient les valeurs des x_i (elle joue le même rôle que la matrice que nous avons notée A). Calculatoirement, la régression de ridge consiste à ajouter $\lambda\,I$ à la matrice de covariance $X^T\,X$ ce qui a pour effet de la rendre définie positive dès que λ est suffisamment grand (les valeurs propres étant augmentées de λ deviennent alors positives) même dans le cas où les colonnes de X sont linéairement dépendantes.

Une méthode similaire à la régression de ridge est employée lors de l'entraînement des réseaux de neurones pour éviter les phénomènes de sur-apprentissage (en Anglais, *overfitting*). Dans ce contexte, elle s'appelle *méthode de dégradation des pondérations* (en Anglais, *weight decay*) [1, section 11.5.2].

Si on remplace la somme de carrés présente dans la contrainte par la somme des valeurs absolues $|\beta_1| + |\beta_2| + \cdots + |\beta_m|$ on obtient une autre méthode appelée *Lasso* [1, section 3.4.2].

6.7.2 Programmation linéaire

> One characteristic of these methods was that they required all iterates to satisfy the inequality constraints in the problem *strictly*, so they soon became known as interior-point methods. By the early 1990s, one class — primal-dual methods — has distinguished itself as the most efficient practical approach and proved to be a strong competitor to the simplex method on large problems.
>
> [2, chapter 14, page 395]

Un autre exemple important vient de la programmation linéaire, en recherche opérationnelle. En la solution optimale d'un programme linéaire primal, les valeurs des multiplicateurs de Lagrange donnent les valeurs des variables du programme linéaire dual [2, chapter 13]. Dans les années 1980, on s'est aperçu qu'on pouvait résoudre un programme linéaire en résolvant directement le système non linéaire donné par les conditions de Karush, Kuhn et Tucker associées au programme. On aboutit ainsi à une famille de méthodes, dites du « point intérieur », qui passent par l'intérieur du polyhèdre des solutions réalisables plutôt que par ses sommets (ce que fait le simplexe).

6.8 Preuve de la proposition 21

Preuve. On suit la démarche suivante : on considère un point $x^* \in \mathbb{R}^m$ et on suppose que le gradient $\nabla f(x^*)$ de l'objectif et celui $\nabla c(x^*)$ de la contrainte ne sont pas parallèles en x^*. On en déduit qu'il est possible de diminuer la valeur de l'objectif tout en respectant la contrainte. L'outil clé que nous allons utiliser est l'inégalité de Cauchy-Schwartz. On cherche donc un vecteur h (une direction dans laquelle se déplacer) tel que $f(x^* + h) < f(x^*)$ et $c(x^* + h) = 0$. La figure 6.2 donne l'intuition dans le cas $m = 2$. Voici un raisonnement général. En supposant h non nul mais très petit, on approxime l'objectif et la contrainte par leur développement limité tronqué à l'ordre un (4.6). Comme f et c sont à image dans $\mathbb{R}$, leur matrice jacobienne est la transposée de leur vecteur gradient et on peut reformuler (4.6) en

$$\begin{aligned} f(x^* + h) &\simeq f(x^*) + (\nabla f)^T(x^*)\, h\,, \\ c(x^* + h) &\simeq c(x^*) + (\nabla c)^T(x^*)\, h\,. \end{aligned}$$

Comme x^* satisfait la contrainte on a $c(x^*) = 0$. Pour que $x^* + h$ satisfasse la contrainte aussi, il faut donc que la direction h soit orthogonale au gradient de la contrainte :

$$(\nabla c)^T(x^*)\, h = 0\,. \tag{6.6}$$

Comme la direction h doit faire diminuer l'objectif, il faut que $f(x^* + h) < f(x^*)$ et donc que la dérivée directionnelle

$$(\nabla f)^T(x^*)\, h < 0\,. \tag{6.7}$$

Montrons que le vecteur h suivant vérifie (6.6) et (6.7). Dans les formules suivantes, on a supprimé les évaluations en x^* pour alléger les notations. Attention à la fraction : le dénominateur est un produit scalaire et donc un réel mais le numérateur est le produit tensoriel de deux vecteurs et donc une matrice.

$$h = -\left(I - \frac{\nabla c\,(\nabla c)^T}{(\nabla c)^T\,\nabla c}\right)\,\nabla f\,. \tag{6.8}$$

Un calcul rapide montre que (6.6) est satisfaite :

$$(\nabla c)^T\, h = -((\nabla c)^T - (\nabla c)^T)\,\nabla f = 0\,.$$

Intéressons-nous maintenant à (6.7). On a

$$(\nabla f)^T\, h = -(\nabla f)^T\,\nabla f + \frac{(\nabla f)^T\,\nabla c\,(\nabla c)^T\,\nabla f}{(\nabla c)^T\,\nabla c}\,, \tag{6.9}$$

$$= -(\nabla f)^T\,\nabla f + \frac{((\nabla f)^T\,\nabla c)^2}{(\nabla c)^T\,\nabla c}\cdot \tag{6.10}$$

Le premier terme du membre droit de (6.10) est un réel négatif (c'est l'opposé d'une somme de carrés). Le deuxième est un réel positif (un carré divisé par une somme de carrés). Il ne reste plus qu'à montrer qu'il est inférieur à la valeur absolue du premier terme. C'est ici qu'on utilise l'hypothèse que les deux vecteurs gradients ne sont pas parallèles : d'après l'inégalité de Cauchy-Schwartz (proposition 3), on a :

$$|(\nabla f)^T\,\nabla c| < \|\nabla f\|_2\,\|\nabla c\|_2\,.$$

En élevant au carré les deux membres de l'inégalité, on a donc

$$((\nabla f)^T\,\nabla c)^2 < (\nabla f)^T\,\nabla f\,(\nabla c)^T\,\nabla c$$

et donc

$$\frac{((\nabla f)^T \nabla c)^2}{(\nabla c)^T \nabla c} \quad < \quad (\nabla f)^T \nabla f$$

ce qui, combiné avec (6.10), prouve l'inégalité (6.7). En résumé, on a prouvé que si les deux gradients ne sont pas parallèles en x^* alors x^* n'est pas une solution optimale. Par conséquent (contraposée), si x^* est une solution optimale alors les deux gradients sont parallèles et la proposition est prouvée. $\square$

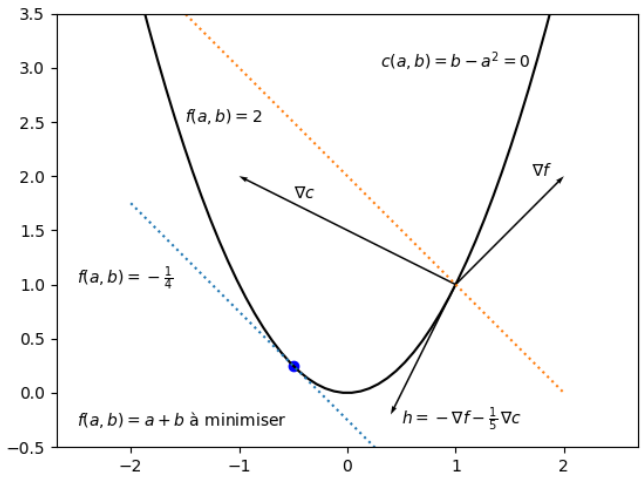


FIGURE 6.2 – Illustration d'un passage de la preuve de la proposition 21. Le problème consiste à minimiser $f(a,b) = a+b$ sous la contrainte $c(a,b) = b-a^2 = 0$ (la solution optimale est $\begin{pmatrix} -\frac{1}{2} & \frac{1}{4} \end{pmatrix}^T$). Intéressons-nous au point $\begin{pmatrix} a^* & b^* \end{pmatrix}^T = \begin{pmatrix} 1 & 1 \end{pmatrix}^T$ qui fait partie des solutions réalisables mais qui n'est pas optimal. Graphiquement, il est facile de trouver un vecteur h (une direction) pour se rapprocher d'une solution optimale : il suffit de suivre la tangente à la contrainte en $\begin{pmatrix} a^* & b^* \end{pmatrix}^T$ dans la bonne direction. C'est le cas du vecteur h du graphique, qui a été obtenu en appliquant la formule (6.8).

Chapitre 7

Retour aux réseaux de neurones

On illustre les notions des sections précédentes sur une variante très simplifiée du problème de reconnaissance des chiffres manuscrits expliqué en section 1.2.

7.1 Un réseau à deux couches

Voir le fichier `7-reseau_deux_couches.py`. Les données d'entraînement de notre réseau sont les suivantes. Disons qu'on cherche un réseau qui reconnaisse les vecteurs commençant par un 1.

a_0	a_1	a_2	y
0	0	1	0
0	1	1	0
1	0	1	1
1	1	1	1

En entrée, quatre vecteurs de dimension 3 (au lieu d'une base de données de vecteurs de dimension 784). En sortie, un réel valant zéro ou un (au lieu d'un vecteur de 10 réels). Le réseau n'a que deux couches (la couche d'entrée et la couche de sortie). Il est représenté schématiquement Figure 7.1. La sortie y se calcule à partir de l'entrée (a_0, a_1, a_2) par la formule suivante, qui dépend de trois poids w_i (on a omis le biais pour simplifier) :

$$y \quad = \quad \sigma(w_0\, a_0 + w_1\, a_1 + w_2\, a_2)\,, \qquad \sigma(x) \quad = \quad \frac{1}{1+e^{-x}}\cdot$$

Pour simplifier l'implantation en Python, au lieu de traiter les $p = 4$ données les unes après les autres, on les traite toutes ensemble (on n'utilise qu'un seul « *batch* » qui contient l'ensemble des données). On définit une matrice A de dimension $p \times m$ où $m = 3$ est le nombre de poids du réseau ainsi qu'un vecteur y contenant les p sorties attendues. La fonction `reseau` prend en entrée le vecteur des poids ainsi qu'une matrice (ou un vecteur ligne) A. Elle retourne le vecteur des valeurs calculées par le réseau, pour chaque ligne de A.

```
import autograd as ag
import autograd.numpy as np

A = np.array ([[0,0,1],[0,1,1],[1,0,1],[1,1,1]], dtype=np.float64)
y = np.array ([0,0,1,1], dtype=np.float64)
```

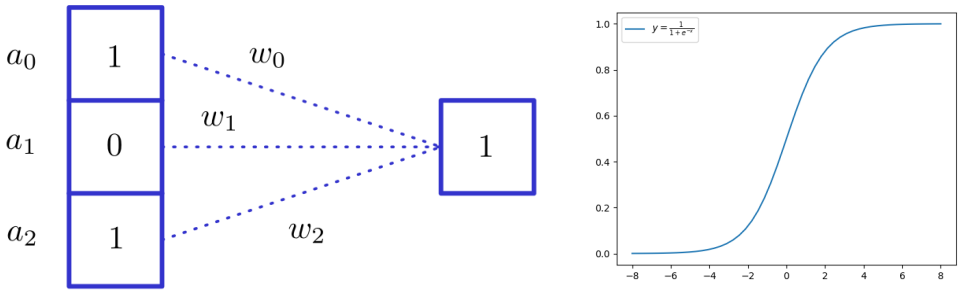


FIGURE 7.1 – À gauche, le réseau de la section 7.1 comporte deux couches et trois poids w_i. À droite, le graphe (une sigmoïde) de la fonction $\sigma(x)$ utilisée par le réseau.

```
p, m = A.shape

def sigmoid (w) :
    return 1. / (1. + np.exp (- w))

def reseau (w, A) :
    l0 = A
    A_w = np.dot (A, w)
    l1 = sigmoid (A_w)
    return l1
```

On peut maintenant introduire la fonction qui, à un vecteur de poids, associe le vecteur des résiduels (les différences entre les sorties attendues et les sorties calculées), l'objectif à minimiser (la somme des carrés des résiduels) ainsi que le gradient de cet objectif. On aurait pu passer A et y en paramètre à ces deux fonctions. On ne le fait pas pour souligner le fait qu'on les voit comme des fonctions des poids.

```
def residuel (w) :
    return y - reseau (w, A)

def objectif (w) :
    r = residuel (w)
    return 1/2 * np.sum (r*r)

g = ag.grad (objectif)
```

On peut maintenant entraîner le réseau : on attribue des valeurs aléatoires comprises entre -1 et 1 aux poids et on effectue 1000 itérations (ou « *epoch* ») de l'algorithme du gradient (sans contrôler le pas).

```
np.random.seed (1)
w = 2 * np.random.random (m) - 1.
for i in range (1000) :
    h = - g (w)
    w = w + h
```

Voici les valeurs retournées par le réseau sur les données d'entraînement. Elles sont assez précises.

```
for i in range (0, p) :
    a = A[i,:] ; print (a, '->', reseau (w, a))

[0. 0. 1.] -> 0.03176744885040881
[0. 1. 1.] -> 0.025751429621741873
[1. 0. 1.] -> 0.9790777923944974
[1. 1. 1.] -> 0.9741600473050827
```

Et voici les valeurs retournées par le réseau sur d'autres données.

```
[1 0 0] -> 0.9992993702390041
[1 1 0] -> 0.9991304666120913
[0 1 0] -> 0.44617285549866925
[0 0 0] -> 0.5
```

Sont-elles précises ? Difficile à dire. Le réseau reconnaît bien les vecteurs qui commencent par un 1 mais il a du mal à rejeter clairement les vecteurs qui commencent par un 0.

7.2 Un réseau à trois couches

Voir le fichier `7-reseau_trois_couches.py`. On cherche maintenant un réseau qui reconnaisse les vecteurs qui ont exactement un 1 en a_0 et a_1. Les données d'entraînement deviennent :

a_0	a_1	a_2	y
0	0	1	0
0	1	1	1
1	0	1	1
1	1	1	0

Ce problème semble plus difficile que le précédent. Le réseau à deux couches se comporte assez mal (essayez !). On peut par exemple essayer d'introduire une couche intermédiaire de $q = 5$ neurones $b_0, \ldots, b_{q-1}$ en utilisant les formules suivantes (toujours sans biais). Voir Figure 7.2.

$$\begin{aligned} b_j &= \sigma(w_{0,j}\, a_0 + w_{1,j}\, a_1 + w_{2,j}\, a_2) \qquad (0 \leqslant j < q)\,, \\ y &= \sigma(w_{m,0}\, b_0 + w_{m,1}\, b_1 + \cdots + w_{m,q-1}\, b_{q-1})\,. \end{aligned}$$

Ces formules impliquent que les poids soient stockés dans une matrice de dimension $(m+1) \times q$. Les m premières lignes servent au calcul de la couche intermédiaire. La dernière ligne sert au calcul de la sortie. Voici le code Python correspondant :

```
A = np.array ([[0,0,1],[0,1,1],[1,0,1],[1,1,1]], dtype=np.float64)
y = np.array ([0,1,1,0], dtype=np.float64)

p, m = A.shape
q = 5

def reseau (w, A) :
    l0 = A
    A_w = np.dot (A, w[0:m,0:q])
    l1 = sigmoid (A_w)
```

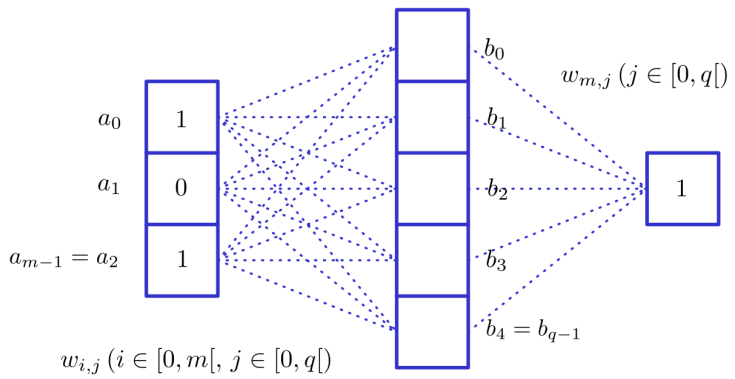


FIGURE 7.2 – Le réseau de la section 7.2 comporte trois couches et 20 poids $w_{i,j}$.

```
    A_w = np.dot (l1, w[m,0:q])
    l2 = sigmoid (A_w)
    return l2

g = ag.grad (objectif)

np.random.seed (1)
w = 2 * np.random.random ((m+1, q)) - 1.
for i in range (500) :
    h = - g (w)
    w = w + h
```

À l'exécution, on voit que le réseau répond assez précisément sur les données d'entraînement.

```
[0. 0. 1.] -> 0.10885055407654746
[0. 1. 1.] -> 0.852382091082654
[1. 0. 1.] -> 0.8361238689012161
[1. 1. 1.] -> 0.2101746957265947
```

Sur les autres données, les résultats sont bons sauf pour le dernier vecteur.

```
[1 1 0] -> 0.24154366385987505
[1 0 0] -> 0.8544574456142336
[0 1 0] -> 0.894948418217487
[0 0 0] -> 0.58425862551317
```

7.3 Commentaires

Sur les deux exemples, on n'a utilisé qu'un seul *batch* contenant toutes les $p = 4$ données d'entraînement. Dans un logiciel de *machine learning*, la taille du *batch* est un paramètre (et même un « *hyperparamètre* ») qui influe sur les performances de l'apprentissage.

D'un point de vue informatique, rien ne nous aurait empêché de tirer aléatoirement un sous-ensemble de ces p données à chaque itération. Il est intéressant de remarquer que le fait de changer le *batch* change la fonction objectif à minimiser (les coefficients de l'objectif sont des fonctions des

données présentes dans le batch). On sort donc un peu du cadre du cours, où la fonction objectif est fixée une fois pour toutes lorsqu'on exécute l'algorithme du gradient ou la méthode de Newton.

On n'a pas contrôlé la longueur du pas, ce qui revient à prendre $\alpha_n = 1$ à chaque itération. Les logiciels de *machine learning* contrôlent la longueur du pas. Le coefficient α_n est fortement lié à un autre *hyperparamètre* appelé « *learning rate* ».

Chapitre 8

Dérivation automatique

La mise en œuvre de la méthode de Newton ou de l'algorithme du gradient est considérablement facilitée par l'utilisation de paquetages de dérivation automatique comme `autograd`. Dans le chapitre 7, on a construit une fonction `g` permettant d'évaluer le gradient de l'objectif à minimiser par un simple appel à la fonction `grad` de ce paquetage.

On présente deux modes de dérivation automatique, tous les deux mis en œuvre dans les logiciels de *machine learning*. Ces deux modes (*forward* et *backward*) calculent les mêmes objets mais pas au même coût. Sur l'exemple de la section 8.2 le mode *backward* utilise huit variables. Sur le même exemple, le mode *forward* utiliserait 2 variables fois 6 nœuds, c'est-à-dire douze variables. Quand on doit calculer le gradient ou la jacobienne d'une fonction $\mathbb{R}^m \to \mathbb{R}^p$, le mode *backward* est plus efficace si $m > p$ alors que le mode *forward* est plus efficace si $m < p$.

On présente les deux modes de dérivation automatique en insérant du code dans un code existant. Les fonctions du paquetage `autograd` ne fonctionnent pas ainsi : elles redéfinissent le type des éléments des tableaux `numpy` et surchargent les opérateurs et les fonctions arithmétiques. Une grande partie du travail est donc effectuée dans une structure de données spécialisée. Toutefois, les enchaînements de calculs sont essentiellement les mêmes que ceux présentés ci-dessous.

8.1 Mode *forward*

La fonction

$$f(a,b) \quad = \quad \sin(a^2) + a^2\, b$$

peut s'évaluer par le code suivant

```
t1 = a*a
t2 = np.sin(t1)
t3 = t1 * b
t4 = t2 + t3
```

Supposons qu'on cherche à calculer son gradient :

$$\nabla f \quad = \quad \begin{pmatrix} \dfrac{\partial f}{\partial a} \\ \dfrac{\partial f}{\partial b} \end{pmatrix} \quad = \quad \begin{pmatrix} 2\,a\,\cos(a^2) + 2\,a\,b \\ a^2 \end{pmatrix}$$

Il suffit d'introduire huit variables

$$\begin{array}{lll} \texttt{dtk_da} & \text{pour} & \dfrac{\partial t_k}{\partial a} \\ \texttt{dtk_db} & \text{pour} & \dfrac{\partial t_k}{\partial b} \end{array} \qquad k = 1, 2, \ldots, 4\,.$$

et d'insérer dans le code initial des instructions qui les calculent. Pour cela, il suffit de voir chacune des variables t_k comme une fonction de a et/ou de b et d'appliquer la formule pour la dérivation (partielle) des fonctions composées. Voici ce que cela donne sur l'exemple. On commence par t_1.

```
t1 = a*a
dt1_da = 2*a
dt1_db = 0
```

Pour calculer `dt2_da`, il faut dériver (par rapport à a) $t_2 = \sin(t_1)$. On rappelle la formule pour dériver les fonctions composées. Notons $(f \circ g)(a) = f(g(a))$. On a :

$$(f \circ g)'(a) \quad = \quad f'(g(a))\, g'(a)\,.$$

Sur l'exemple, nous avons $f = \sin$, $g = t_1$ et donc $f' = \cos$. La dérivation est la dérivation partielle par rapport à a donc $g' = \frac{\partial t_1}{\partial a}$. Or nous avons déjà calculé cette quantité et nous l'avons stockée dans `dt1_da`. Nous obtenons donc (le calcul de `dt2_db` est similaire) :

```
t2 = np.sin(t1)
dt2_da = np.cos(t1) * dt1_da
dt2_db = np.cos(t1) * dt1_db
```

Pour calculer `dt3_da`, il faut dériver (par rapport à a) $t_3 = t_1\, b$. On a $f : x \mapsto b\, x$, $g = t_1$ et donc $f' = b$. On obtient :

```
t3 = t1 * b
dt3_da = dt1_da * b
```

Pour calculer `dt3_db`, il faut dériver (par rapport à b) $t_3 = t_1\, b$. On applique la formule pour la dérivée d'un produit : $\frac{\partial t_3}{\partial b} = \frac{\partial t_1}{\partial b}\, b + t_1$ qui se traduit par :

```
dt3_db = dt1_db * b + t1
```

Même principe en plus simple pour t_4. Les variables `dt4_da` et `dt4_db` contiennent les deux coordonnées du gradient.

```
t4 = t2 + t3
dt4_da = dt2_da + dt3_da
dt4_db = dt2_db + dt3_db
```

8.2 Mode *backward*

On cherche à calculer le gradient ∇f de la fonction suivante par une méthode algorithmique :

$$f(a, b) \quad = \quad (e^a)^2 + a\, b\, e^a + \sin(e^a)\,.$$

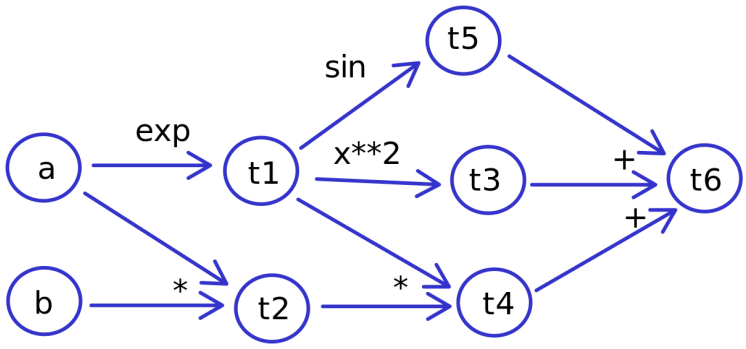
Le résultat attendu est

$$\nabla f \quad = \quad \begin{pmatrix} \dfrac{\partial f}{\partial a} \\ \dfrac{\partial f}{\partial b} \end{pmatrix} \quad = \quad \begin{pmatrix} 2\,(e^a)^2 + (a\,b + \cos\,(e^a) + b)\;e^a \\ a\,e^a \end{pmatrix}.$$

Le code suivant évalue f :

```
t1 = exp(a)
t2 = a*b
t3 = t1**2
t4 = t1*t2
t5 = sin(t1)
t6 = t3 + t4 + t5
```

On peut lui associer le graphe suivant :



Considérons un nœud (disons) t_1. Prenons tous les nœuds qui l'utilisent explicitement (les successeurs de t_1 dans le graphe) : t_3, t_4 et t_5. À partir des instructions, il est facile de calculer $\frac{\partial t_k}{\partial t_1}$ pour $k = 3, 4, 5$:

$$\begin{array}{lll} t_3 = t_1^2 & \dfrac{\partial t_3}{\partial t_1} & = \quad 2\,t_1\,, \\ t_4 = t_1\,t_2 & \dfrac{\partial t_4}{\partial t_1} & = \quad t_2\,, \\ t_5 = \sin(t_1) & \dfrac{\partial t_5}{\partial t_1} & = \quad \cos(t_1)\,. \end{array}$$

On peut donc évaluer partiellement la formule suivante (pour une justification, voir la section 8.2.1) :

$$\frac{\partial f}{\partial t_1} \quad = \quad \frac{\partial f}{\partial t_3}\,\frac{\partial t_3}{\partial t_1} + \frac{\partial f}{\partial t_4}\,\frac{\partial t_4}{\partial t_1} + \frac{\partial f}{\partial t_5}\,\frac{\partial t_5}{\partial t_1}\,, \tag{8.1}$$

$$\quad = \quad 2\,t_1\,\frac{\partial f}{\partial t_3} + t_2\,\frac{\partial f}{\partial t_4} + \cos(t_1)\,\frac{\partial f}{\partial t_5}. \tag{8.2}$$

Et on voit que, si les valeurs des $\frac{\partial f}{\partial t_k}$ sont stockées dans des variables `df_dtk` pour $k = 3, 4, 5$ et t_2 est évaluée alors il est facile d'obtenir $\frac{\partial f}{\partial t_1}$. La formule (8.1) peut se formuler de façon générale ainsi :

$$\frac{\partial f}{\partial t} \quad = \quad \sum_{s \text{ successeur de } t} \frac{\partial f}{\partial s}\,\frac{\partial s}{\partial t}. \tag{8.3}$$

Le principe de la dérivation automatique (*backward mode*) consiste à évaluer la fonction f complètement (on est alors sûr que toutes les variables t_i sont évaluées) puis calculer les valeurs des $\frac{\partial f}{\partial t_k}$ en partant de la fin du graphe (pour $k = 6, 5, \ldots, 1$, donc) et en finissant par $\frac{\partial f}{\partial a}$ et $\frac{\partial f}{\partial b}$ (les valeurs recherchées). Plus généralement, il faut énumérer les nœuds du graphe suivant un ordre topologique. Sur l'exemple, on s'assure ainsi que les valeurs des $\frac{\partial f}{\partial t_k}$ pour $k = 3, 4, 5$ sont évaluées avant d'évaluer $\frac{\partial f}{\partial t_1}$. On obtient :

```
t1 = exp(a)
t2 = a*b
t3 = t1**2
t4 = t1*t2
t5 = sin(t1)
t6 = t3 + t4 + t5
df_dt6 = 1
df_dt5 = df_dt6
df_dt4 = df_dt6
df_dt3 = df_dt6
df_dt2 = t1*df_dt4
df_dt1 = 2*t1*df_dt3 + t2*df_dt4 + cos(t1)*df_dt5
df_da = exp(a)*df_dt1 + b*df_dt2
df_db = a*df_dt2
```

8.2.1 Justification de la formule (8.1)

Il suffit de savoir que si f est une fonction de trois variables a, b et c alors la *différentielle* de f s'écrit :

$$\mathrm{d}f = \frac{\partial f}{\partial a}\,\mathrm{d}a + \frac{\partial f}{\partial b}\,\mathrm{d}b + \frac{\partial f}{\partial c}\,\mathrm{d}c\,.$$

La formule se généralise à n variables, bien sûr.

Revenons à la fonction f de l'exemple. On peut la voir comme une fonction $f(t_3, t_4, t_5)$. On a donc

$$\mathrm{d}f = \frac{\partial f}{\partial t_3}\,\mathrm{d}t_3 + \frac{\partial f}{\partial t_4}\,\mathrm{d}t_4 + \frac{\partial f}{\partial t_5}\,\mathrm{d}t_5\,. \tag{8.4}$$

Les variables t_3 et t_5 sont des fonctions de t_1. La variable t_4 est une fonction de t_1 et de t_2. On a donc

$$\begin{aligned}
\mathrm{d}t_3 &= \frac{\partial t_3}{\partial t_1}\,\mathrm{d}t_1\,,\\
\mathrm{d}t_4 &= \frac{\partial t_4}{\partial t_1}\,\mathrm{d}t_1 + \frac{\partial t_4}{\partial t_2}\,\mathrm{d}t_2\,,\\
\mathrm{d}t_5 &= \frac{\partial t_5}{\partial t_1}\,\mathrm{d}t_1\,,
\end{aligned}$$

En remplaçant les variables $\mathrm{d}t_k$ par leur valeur dans (8.4) (pour $k = 1, 2, 3$) on obtient

$$\mathrm{d}f = \frac{\partial f}{\partial t_3}\frac{\partial t_3}{\partial t_1}\,\mathrm{d}t_1 + \frac{\partial f}{\partial t_4}\left(\frac{\partial t_4}{\partial t_1}\,\mathrm{d}t_1 + \frac{\partial t_4}{\partial t_2}\,\mathrm{d}t_2\right) + \frac{\partial f}{\partial t_5}\frac{\partial t_5}{\partial t_1}\,\mathrm{d}t_1\,, \tag{8.5}$$

$$= \left(\frac{\partial f}{\partial t_3}\frac{\partial t_3}{\partial t_1} + \frac{\partial f}{\partial t_4}\frac{\partial t_4}{\partial t_1} + \frac{\partial f}{\partial t_5}\frac{\partial t_5}{\partial t_1}\right)\mathrm{d}t_1 + \frac{\partial f}{\partial t_4}\frac{\partial t_4}{\partial t_2}\,\mathrm{d}t_2 \tag{8.6}$$

Mais la fonction f est aussi une fonction $f(t_1, t_2)$ et donc

$$\mathrm{d}f \quad = \quad \frac{\partial f}{\partial t_1}\,\mathrm{d}t_1 + \frac{\partial f}{\partial t_2}\,\mathrm{d}t_2\,. \tag{8.7}$$

En identifiant les coefficients de $\mathrm{d}t_1$ et $\mathrm{d}t_2$ dans (8.6) et (8.7) on trouve (la première ligne correspond à la formule (8.1)) :

$$\begin{aligned}
\frac{\partial f}{\partial t_1} &= \frac{\partial f}{\partial t_3}\,\frac{\partial t_3}{\partial t_1} + \frac{\partial f}{\partial t_4}\,\frac{\partial t_4}{\partial t_1} + \frac{\partial f}{\partial t_5}\,\frac{\partial t_5}{\partial t_1}\,, \\
\frac{\partial f}{\partial t_2} &= \frac{\partial f}{\partial t_4}\,\frac{\partial t_4}{\partial t_2}
\end{aligned}$$

Index

algorithme du gradient, 4, 29
`autograd`, 43, 46

backtrack line search, 30

Cauchy-Schwartz, 5, 29, 39
col, 28
contrainte active, 37
contraintes d'un problème d'optimisation, 37
contribution des termes d'ordre deux, 28
contrôle du pas, 30

dérivation automatique, 46
dérivée directionnelle, 29
dérivée partielle, 21
descente, 29
différentielle, 34, 49

écriture matricielle du développement limité, 22
équations normales, 9
espace tangent, 33

factorisation $Q\,R$, 10
factorisation de Cholesky, 9, 28

`grad`, 43, 46
gradient orthogonal aux courbes de niveau, 33
gradients parallèles, 35

H_f, 26
hypothèse d'indépendance linéaire, 9

KKT, 37

Lagrangien, 35, 37
line search, 32

matrice définie positive, 6
matrice hessienne, 26
matrice hessienne définie positive, 27
matrice hessienne jacobienne du gradient, 26
matrice jacobienne, 22
matrice jacobienne d'un objectif, 25
matrice orthogonale, 5
maximum local, 28
méthode de Newton, 4
méthode dichotomique, 16
minimum local, 4, 28
minimum local (une variable), 18
moindres carrés, 38
moindres carrés linéaires, 8
moindres carrés non linéaires, 4
multiplicateur de Lagrange, 34

∇f, 22
$\nabla^2 f$, 26
neurone, 4
Newton, 23, 29
norme deux, 5

objectif, 4
objectif d'un problème d'optimisation, 25
overfitting, 38

phénomène non intuitif, 28
plan tangent, 25
plus grande pente, 29
point stationnaire, 25, 35

quadratique, 16, 23

racine double, 16
racine simple, 16
rang maximal, 9
recherche linéaire, 32
région de confiance, 32
régression de ridge, 38
régression du Lasso, 38
réseau de neurones, 3, 41

résiduel, 4, 8

`sympy`, 16

tangente et espace tangent, 34
trust region, 32

valeurs propres d'une matrice définie positive, 6
valeurs propres d'une matrice symétrique, 5
valeurs propres de la matrice hessienne, 28
valeurs singulières d'une matrice, 9
vecteur gradient, 22
vecteurs orthogonaux, 5
vecteurs parallèles, 35
vitesse de convergence, 16, 23

weight decay, 38

Bibliographie

[1] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning. Data Mining, Inference and Prediction.* Springer Series in Statistics. Springer, 2nd edition, 2009. Available at `https://hastie.su.domains/ElemStatLearn`.

[2] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization.* Springer Series in Operation Research. Springer Verlag, 1999.

[3] Grant Sanderson. Gradient descent, how neural networks learn | Chapter 2, Deep learning. `https://www.youtube.com/watch?v=IHZwWFHWa-w&t=2s`, 2020.

[4] Lloyd Nicholas Trefethen and David Bau. *Numerical Linear Algebra.* SIAM, 1997.